

QuantumTrustKG: Provenance-Aware Orchestration for Quantum-Safe Microservices

Xuan-Bach Le¹ ^{✉*}, Phat T. Tran-Truong¹ , Duy Nguyen² , Ngoc Khanh
Tran Nguyen³ , and Xuan Son Ha⁴ 

¹ Ho Chi Minh City University of Technology (HCMUT), VNU-HCM, Vietnam
`{lexuanbach,phatttt}@hcmut.edu.vn`

² University of Economics Ho Chi Minh City, Vietnam
`duynguyen.726202321440@st.ueh.edu.vn`

³ University of Science, VNU-HCM, Vietnam
`22120378@student.hcmus.edu.vn`

⁴ RMIT University, Vietnam
`ha.son@rmit.edu.vn`

Abstract. Post-quantum migration requires a service mesh to select cryptographic profiles as endpoint capabilities, policies, threats, and evidence freshness change. Static mesh settings and policy-as-code can enforce a selected posture but do not perform this edge-specific selection. We present *QuantumTrustKG*, a provenance-aware, graph-backed control-plane framework. For each communication edge, its Quantum Trust Protocol Orchestration (QTPO) procedure removes profiles that fail a capability, policy, threat, or freshness gate, ranks the remaining profiles by cost, and blocks promotion if none is admissible. We mechanize the control-plane guarantees in Coq/Rocq and implement QTPO as a Kubernetes controller integrated with Istio. In a controlled comparison on 200-service meshes, QTPO records no unsafe promotions and 96.5% *declared-label policy compliance*. Service-level and opportunistic baselines produce unsafe promotions on 3.4–13.7% of edges. On sampled production service graphs of up to 2000 services, service-level and per-endpoint baselines reach 37–60%, while QTPO remains at zero. The compliance metric is computed from the declared labels used by the controller. It therefore measures control-plane decisions given those inputs rather than the cryptography negotiated on the wire. Extended version of the ICSOC 2026 paper. Artifact: <https://github.com/lexuanbach/quantumtrustkg>.

Keywords: service orchestration · microservices · knowledge graphs · service mesh · post-quantum cryptography · crypto-agility

1 Introduction

Organizations are beginning to migrate to post-quantum cryptography (PQC) to address store-now-decrypt-later attacks [44,29]. The transition is gradual:

* X.-B. Le and P. T. Tran-Truong contributed equally to this work.

✉ Corresponding author: Xuan-Bach Le, `lexuanbach@hcmut.edu.vn`.

endpoints gain support at different times, policies vary across trust boundaries, and threat information expires. Connections carrying regulated or long-lived data may consequently need quantum-safe protection while adjacent legacy endpoints or partner gateways remain on transitional profiles [3,5,4]. For example, a checkout service may require a quantum-safe profile for its regulated card-processor edge while retaining a hybrid profile for a cache whose peer is still migrating (Fig. 1, stage 4). A single service-level posture cannot express both requirements.

Service meshes enforce configured profiles, and policy-as-code validates those configurations [11]. Selecting a profile as capabilities, policies, advisories, and rollout state change requires a separate control loop. We introduce *QuantumTrustKG*, a control-plane framework and Kubernetes/Istio controller that performs this selection for each communication edge. A knowledge graph (KG) joins services, edges, profiles, policies, provenance, and rollout state. The Quantum Trust Protocol Orchestration (QTPO) procedure removes every candidate that fails a capability, policy, threat, freshness, or rollout gate and ranks the remaining profiles by cost. If no profile is admissible, the controller preserves the last committed configuration, blocks the new assignment, and records the failed gate.

We design the decision layer around three properties. First, QTPO assigns profiles per edge. Existing per-service governance [34], per-endpoint profiling [35], and static inventories [41] cannot represent different obligations on two inbound edges of the same service. Istio can scope configuration to a workload, port, or destination, but it does not infer a posture from a changing source–destination context. Section 8 evaluates the resulting granularity gap on synthetic and production topologies.

Second, admissibility is independent of cost. The ranker receives only candidates that passed the hard gates. Changing the score weights cannot promote an inadmissible profile (Prop. 2). This differs from quality-of-service-aware service selection, which ranks candidates under a fixed correctness posture [18,16,17,19]. Third, provenance participates in admission. Stale, contradictory, or unavailable evidence removes a candidate instead of merely lowering its score. Static mesh configuration has no representation of fact age, while conventional policy-as-code consumes assertions without maintaining their source and freshness window. We mechanize the admission and fail-closed properties in Coq/Rocq (Sec. 6).

We evaluate control-plane decisions with *declared-label policy compliance* (C_{decl}), computed from workload capability claims, operator policies, and profile classes recorded in the same metadata consumed by the controller. A workload that declares quantum-safe support but negotiates classical Transport Layer Security (TLS) is therefore still counted as compliant. The metric measures whether a control-plane decision satisfies its declared inputs. It is not evidence of the cryptography negotiated on the wire. Sections 10 and 12 make this boundary explicit.

We make four contributions. We formulate PQC migration as an edge-level constrained selection problem (Sec. 2), develop the graph-backed evidence model and Kubernetes/Istio backend (Sec. 4), present QTPO and its mechanized control-plane guarantees (Secs. 5–6), and evaluate the prototype on 50–200-service

testbeds, sampled Alibaba production topologies, and semantic-core stress tests up to 2000 services (Secs. 7–8).

2 Background and the Edge-Level Decision Problem

PQC, crypto-agility, and the mesh. Post-quantum cryptography (PQC) denotes key-establishment and signature algorithms that resist large-scale quantum computers. The US National Institute of Standards and Technology (NIST) standardized the first of them as Federal Information Processing Standards (FIPS) 203–205: the Module-Lattice-Based Key-Encapsulation Mechanism (ML-KEM) and the Module-Lattice- and Stateless-Hash-Based Digital Signature Algorithms (ML-DSA, SLH-DSA) [29,1]. Because an adversary can harvest today’s traffic and decrypt it later, regulated and long-lived flows migrate first—but never all at once. Endpoints, gateways, and dependencies acquire PQC support at different times. A deployment passes through a transitional phase in which *hybrid* profiles (a classical and a post-quantum key exchange run together [30,31]) coexist with purely classical and purely quantum-safe ones. The engineering goal is *crypto-agility*: adopting, rotating, and mixing profiles without re-architecting services [2].

A service mesh such as Istio already manages workload identity, mutual Transport Layer Security (mTLS), policy distribution, and observability [7,5], and a `DestinationRule` or `PeerAuthentication` resource sets the TLS mode on a connection. The mesh is therefore the natural place to apply ordinary mTLS policy. In this prototype the selected PQ profile is desired-state metadata. Stock Envoy/Istio does not negotiate the corresponding group. Policy-as-code engines such as the Open Policy Agent (OPA) sit alongside it and validate or reject configuration against written rules. What neither offers is a way to decide *which* profile is justified for one connection, because that depends on per-connection obligations, trust boundaries, and the migration state of both endpoints—evidence that a mesh resource, however finely scoped, does not itself weigh.

The edge-level decision problem. For a migration controller the unit of work is a logical service-communication edge at reconciliation rather than an individual RPC or TLS connection. We model the application as a directed service graph $G = (\mathcal{S}, E)$, in which $e = (s_i, s_j)$ is a call from s_i to s_j . Services publish capabilities, labels, obligations, and migration state. Edges carry the context that makes selection local: the trust boundary crossed, the active policy, the current assignment, the rollout status, and, for each fact, its provenance—the source it came from and the window over which it remains fresh. Acting on a stale or contradictory fact can silently weaken protection. The controller treats these facts as *evidence* rather than as authority.

A concrete edge makes the local choice easier to see (Fig. 2, Sec. 7). In the Online Boutique fixture [26], the `Checkout-to-Payment` edge inherits Payment Card Industry (PCI) policy and a regulated boundary. Hybrid protection is insufficient once a `quantum_safe` profile is available at both endpoints. A neighbouring edge

stays hybrid when policy permits transition and one endpoint is still migrating. Edge by edge, the controller chooses between promotion, preservation, and an explicit block: a promotion rewrites that connection’s mesh configuration (its `DestinationRule/PeerAuthentication`) and records the decision trace, while a block preserves the last good configuration and records which fact, policy, or rollout condition prevented a new assignment. A blocked promotion does not deny traffic: the previous `activeProfile` can remain, while `trafficDenied` stays false unless a separate deny mechanism is configured. In both cases the controller first joins the provenance-qualified evidence for the edge, and what a promotion changes is the profile recorded in the edge’s Istio objects: stock Istio enforces ordinary mTLS and carries that profile as control-plane metadata, and enforcing the declared post-quantum group on the wire needs a post-quantum-capable data plane (Appendix C).

Let $\mathcal{P}$ be the set of deployable protocol profiles. A profile p records its key-establishment and authentication families, its security assumption, its proof status, its deployment cost, and its declared security class:

$$\text{Security}(p) \in \{\text{classical}, \text{hybrid}, \text{quantum_safe}\}, \quad \text{classical} \preceq \text{hybrid} \preceq \text{quantum_safe}.$$

These ordered values are operator policy classes rather than a universal ranking of cryptographic strength. This metadata is evidence for policy and audit logic. It is a control-plane guarantee rather than a claim about the underlying primitive: the class is *declared* by whoever registered the profile, and the controller reasons over that declaration.

A controller assignment is a function $\pi : E \rightarrow \mathcal{P} \cup \{\text{blocked}\}$ mapping each edge to a profile or to a block. A promoted edge ($\pi(e) \in \mathcal{P}$) is valid only if its profile is supported by both endpoints, satisfies the edge’s required security class $\text{Req}(e)$, respects proof-status and family restrictions, rests on fresh and non-contradictory facts, and is rollout-feasible. It is *compliant* when $\text{Security}(\pi(e)) \succeq \text{Req}(e)$, the one exception being an explicitly policy-permitted hybrid fallback. No-promotion and retained-active outcomes are reported separately and never counted as promotions.

The trust model sits at the control plane. Kubernetes, Istio, and the Custom Resource Definition (CRD) store are trusted. Capability claims and advisories are observations qualified by source and freshness. A stale or contradictory fact can block or preserve an assignment but can never justify a weaker one. Insider cases beyond role-based access control (RBAC)—capability spoofing and annotation tampering by an authorized principal—are out of scope, and §10 states precisely what the guarantees do and do not cover.

The orchestration problem is then to compute π so that every promoted edge is admissible, provenance-valid, and deployable, while preferring lower cost: $\min_{\pi} \sum_{e: \pi(e) \in \mathcal{P}} \text{Overhead}(\pi(e))$. We solve it with an edge-local heuristic that is lexicographic in spirit, ensuring that validity and deployability dominate cost rather than trading against it.

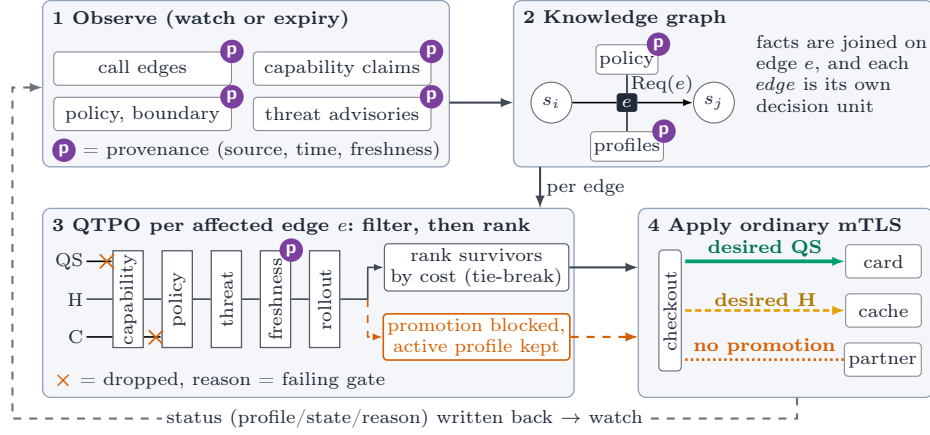


Fig. 1. The QuantumTrustKG control loop. QTPO filters each edge’s classical (C), hybrid (H) and quantum-safe (QS) candidates and ranks the survivors by cost. One **checkout** service receives three per-edge outcomes.

3 QuantumTrustKG

We implement the control loop in Fig. 1 as Algorithm 1. On each watch event or scheduled evidence expiry, the controller normalizes topology, capability claims, policies, trust boundaries, advisories, and rollout state into the Quantum Trust Graph. QTPO then filters and ranks profiles for each affected edge, materializes accepted assignments as Istio configuration, and writes auditable status. Attaching the decision to an edge lets one **checkout** service take three different postures on three calls (Fig. 1, stage 4).

The loop has three operational properties. It is *incremental*: only edges touched by the event are re-evaluated. It is *fail-closed*: if fresh evidence cannot establish admissibility, including when the graph store is unreachable, the controller preserves the previous assignment instead of selecting a weaker one. It is also *off the data path*: selection runs on configuration changes. Its cost is a control-loop cost rather than per-connection latency. Fail-closed here means no new promotion: it does not prove that the retained active profile remains admissible or that traffic is denied.

4 The Semantic Control-Plane Model

We use the Quantum Trust Graph to join topology, policy, capability, provenance, and rollout evidence before a promotion is decided. It holds services, communication edges, protocol profiles, policies, trust zones, provenance observations, rollout state, and the desired and active assignments. Its central object, `:CommunicationEdge`, is where endpoint capabilities, inherited policy,

Algorithm 1: QuantumTrustKG edge reconciliation on a watch event or evidence expiry.

Input: cluster snapshot; watch-affected edges E'
Output: mesh config + **AssignmentStatus** per edge

```

1  $G \leftarrow \text{SyncGraph}(\text{services, capabilities, policies, advisories, rollout})$  // write to Fuseki
2 foreach  $e \in E'$  do
3   if  $\neg \text{GraphAvailable}()$  then preserve  $\pi(e)$ ; MARKBLOCKED( $e$ , graph-unavailable);
   continue // fail-closed (G4)
4    $C \leftarrow \text{CompatibleProfiles}(e)$  // both endpoints, fresh records; else blocked
5    $A \leftarrow \text{FilterGates}(C, e, \text{Req}(e))$  // class, threat, freshness, contradiction, rollout
6   if  $A \neq \emptyset$  then  $p^* \leftarrow \arg \min_{p \in A} \text{Score}(p, e)$  // rank survivors by cost
7   else if  $\text{HybridPermitted}(e) \wedge \exists$  admissible robust hybrid  $h$  then  $p^* \leftarrow h$  // guarded fallback
8   else MARKBLOCKED( $e$ , failed gate); continue // no promotion; retain active profile
9   APPLY( $\text{PeerAuthentication}(e), \text{DestinationRule}(e)$ ) for  $p^*$  // mTLS objects, profile as metadata
10  if  $\text{BothWorkloadsReady}(e)$  then  $\pi(e) \leftarrow p^*$ ; WRITESTATUS( $e$ ) // commit after readiness
11  else keep  $\pi(e)$ ; mark Applying; REQUEUE( $e$ ) // previous profile stays active
12 end

```

trust-boundary context, protocol evidence, provenance, and rollout state are joined at the point of decision.

Policy representation. A policy matches an edge through service selectors, trust-boundary labels, and compliance context. When several policies match one edge, we collapse all matches to the strongest required class:

$$\text{Req}(e) = \max_{\preceq} \{ \ell \mid \text{requiresLevel}(\rho, \ell) \wedge \rho \text{ matches } e \}.$$

A permissive or lower-cost rule therefore cannot weaken a stricter applicable requirement. Deprecated primitive families, proof-status restrictions, and active threat advisories are *hard gates*: they block promotion before any ranking happens. Hybrid fallback is entered only when the active policy explicitly permits it and no stronger quantum-safe candidate is feasible.

Fact provenance. Every fact records a source, an observation time, a freshness window, and any contradiction notes. A fact f is fresh at reconciliation time t only when $t \leq \text{observedAt}(f) + \text{freshnessWindow}(f)$. Capability CRDs default to a 300s window. Expired or contradictory facts, such as conflicting live capabilities or active advisories, block a new promotion and never justify a weaker assignment. QTPO reads the graph through four queries: edge requirements, endpoint/profile compatibility, direct-PQC or hybrid-transition feasibility, and stale or contradictory observations.

Graph-backend rationale. We store the graph as the Resource Description Framework (RDF) on Apache Jena Fuseki and query it with the SPARQL Protocol and RDF Query Language (SPARQL) [46]. Three properties of the problem drove that choice. The inputs are *heterogeneous and open*—topology, capability claims, policy CRDs, advisories, rollout state, with no shared schema and new fact types appearing over time—so a triple model absorbs a new predicate without

a relational migration. The decisions are *late multi-way joins*, which SPARQL expresses directly. And *provenance must travel with the fact*: RDF named graphs attach a source and a freshness window to each individual statement. A promotion can cite the statements that justify it. A labeled-property graph serves the joins but offers weaker per-statement provenance, and a relational design re-introduces the schema rigidity we are trying to avoid.

We quantify the query cost with a same-join microbenchmark (`make datamodel-benchmark`, Table A4) runs the identical per-edge admissibility join over the 1000-service fixture under three encodings. A property-graph (networkx) encoding answers in ≈ 6.8 ms and a relational (SQLite) one in ≈ 9.7 ms, against ≈ 1.0 s for rdflib/SPARQL—all three returning the same 1889 admissible edges. The 1 s figure is rdflib, a pure-Python reference library used only for this comparison. The deployed store is Fuseki, against which the 12–45 ms selection latency of Sec. 8 is measured. And because selection runs off the data path, a control loop can absorb a cost that a per-request path could not. At runtime the controller combines SPARQL joins with explicit Go checks, and Shapes Constraint Language (SHACL) files validate fixtures rather than acting as the controller’s semantics.

Contribution of the graph model. The graph’s value is that it keeps the facts a promotion needs together until reconciliation, instead of flattening them into per-edge records at ingestion time. The QTPO-NoKG ablation removes late joins, provenance locality, freshness, contradiction detection, and decision traces, while keeping the same filters, weights, and hysteresis. It is therefore the comparison in which the only difference is the graph-backed model (Sec. 8.4).

5 Protocol Selection with QTPO

We turn graph state into a deployable action for one edge by *filtering before it ranks*. For that edge it computes the profiles both endpoints support (COMPATIBLEPROFILES), deletes every candidate that fails a policy, threat, provenance, or rollout gate (FILTERGATES), and compares only the survivors by cost ($\arg \min \text{Score}$), committing the winner only once its mesh objects and both workloads are ready. If no candidate survives, it tries a hybrid fallback—but only when the direct admissible set is empty and policy explicitly permits one—and otherwise blocks promotion with the failed gate as the recorded reason. The result is the assignment $\pi : E \rightarrow \mathcal{P} \cup \{\text{blocked}\}$.

Algorithm 2 gives the procedure in full. In its filter-before-rank order, FILTERGATES decides *admissibility* against five independent hard gates, and only survivors reach the cost ranker. Because the ranker never sees an inadmissible profile, changing its weights can change *which* safe profile is promoted, but can never turn an inadmissible profile into a promotion. The safety and fail-closed guarantees of Sec. 6 are consequently weight-independent.

Algorithm details. Line 1 resolves $\text{Req}(e)$, the strongest security class among all policies applicable to e and the trust-boundary floor. A permissive rule cannot weaken a stricter one. Line 2 restricts candidates to C_e , the profiles that *both*

Algorithm 2: QTPO: per-edge protocol selection.

Input: Quantum Trust Graph G ; edge $e = (s_i, s_j)$; reconciliation time t
Output: assignment $\pi(e) \in \mathcal{P} \cup \{\text{blocked}\}$

```

1  $R \leftarrow \text{Req}(e)$  // strongest applicable policy class (§4)
2  $C_e \leftarrow \text{CompatibleProfiles}(s_i, s_j)$  // advertised by both endpoints
   (fresh records)
3  $A \leftarrow \{p \in C_e \mid \text{FilterGates}(p, e, t)\}$  // keep only admissible profiles
4 if  $A = \emptyset$  and  $\text{PolicyPermitsHybrid}(e)$  then
5   |  $A \leftarrow \{h \in C_e \mid h \text{ hybrid and robust,}$ 
   |    $\text{FilterGates}(h, e, t) \text{ under fallback policy}\}$  // guarded fallback
6 end
7 if  $A = \emptyset$  then return  $\pi(e) \leftarrow \text{blocked}$  // fail closed; reason = first
   failed gate
8  $p^* \leftarrow \arg \min_{p \in A} \text{Score}(p, e)$ ; apply  $\text{PeerAuthentication/DestinationRule}$ 
   // cost tie-break among safe choices
9 if  $\text{MeshReady}(e, p^*)$  then return  $\pi(e) \leftarrow p^*$  // both workloads ready:
   commit as active
10 return  $\pi(e) \leftarrow \text{blocked}$  // not ready: last committed profile stays;
   retry

11 Function  $\text{FilterGates}(p, e, t)$ :
12   if  $\text{Security}(p) \not\geq R$  then return false // (i) policy class gate
13   if  $\text{deprecated}(\text{fam}(p)) \vee \neg \text{proofStatusOK}(p)$  then return false // (ii)
   family / proof status
14   if  $\exists \text{ active advisory against fam}(p)$  then return false // (iii) threat
   gate
15   if  $\exists f \in \text{facts}(p, e) : \neg \text{fresh}(f, t) \vee \text{contradicted}(f)$  then return false
   // (iv) provenance
16   if  $\neg \text{rolloutFeasible}(p, e)$  then return false // (v) rollout gate
17   return true

```

endpoints advertise support for in fresh capability records. A profile no peer can negotiate is never considered, and a missing or stale record or an empty intersection leaves $C_e = \emptyset$. The edge blocks. Line 3 applies the five gates. Each is a hard constraint: (i) the profile's declared class must meet or exceed R , (ii) its primitive family must be neither deprecated nor of insufficient proof status, (iii) no active advisory may target that family, (iv) every provenance fact the decision reads must be fresh ($t \leq \text{observedAt} + \text{freshnessWindow}$) and non-contradictory, and (v) the profile must be rollout-feasible, meaning promotable without forcing a peer into a mode it does not support. A candidate that fails any gate is dropped, with that gate recorded as its rejection reason. The survivors are ranked by Score and the minimum-cost profile is committed as active only once its Istio objects exist and both workloads are ready. Until then the previous profile stays active and the edge is retried. When no candidate survives, a hybrid profile is admitted only if the active policy explicitly permits fallback, the hybrid is recorded as a robust combiner, *and* it clears every gate. Otherwise the edge is blocked, the

failed gate is recorded, and the last committed assignment is preserved. QTPO never downgrades silently.

Cost-based ranking. Among admissible candidates we choose the lowest score:

$$\text{Score}(p, e) = w_o \text{Overhead}(p) + w_r \text{Risk}(p, e) + w_m \text{MigrationCost}(p, e) + w_h \text{SwitchPenalty}(p, p_{\text{curr}}(e)).$$

The four terms are Overhead (setup and wire cost), Risk (residual exposure), MigrationCost (rollout disruption), and SwitchPenalty (movement away from the currently active profile). Our defaults are $w_o=0.25$, $w_r=0.40$, $w_m=0.20$, $w_h=0.15$, with hysteresis $\delta=0.05$ (an absolute score gap) damping capability churn. The controller and every harness read them from one typed configuration, and Sec. 10 gives operator guidance on setting them. Sweeping each weight and δ by $\pm 50\%$ on the 200-service H2 meshes changes the promoted profile on up to 20.5% of promoted edges but leaves $C_{\text{decl}}^{\text{harness}}$ at 96.5% and unsafe promotions at zero [M] (Sec. 8.8, and cf. Prop. 2): admissibility, and therefore safety, is weight-independent. The sweep only trades one admissible profile for another. A graph-valid profile is committed only after its Istio objects and both workloads are ready.

Risk term. Risk is a normalized residual-exposure penalty in $[0, 1]$ computed from three inputs: the class gap between $\text{Security}(p)$ and $\text{Req}(e)$, boundary sensitivity s_b (regulated $1.0 > \text{partner } 0.6 > \text{internal } 0.3$), and the maximum severity a_p of any non-blocking advisory against p 's family. A candidate below $\text{Req}(e)$ scores 1 (gate (i) has already removed it. The value appears only in decision traces). Otherwise $\text{Risk}(p, e) = \min\{1, s_b \cdot d(p) + a_p\}$, where $d(p)$ is p 's distance to `quantum_safe` ($0, \frac{1}{2}$ or 1 class steps normalized). It is therefore 0.15 for a hybrid on an internal edge and 0 for any quantum-safe profile. Raising w_r favours stronger admissible profiles. Since admissibility is fixed *before* scoring (Prop. 2), the weights only reorder admissible candidates. We fix interpretable defaults and leave learned weighting to future work.

6 Guarantees

Let $A_{\text{dir}}(e, t)$ be the set of *directly admissible* profiles for edge e at reconciliation time t . The guarantees follow from the controller's filter order and promotion rule rather than from a cryptographic reduction. They also assume that the external cryptographic claims attached to each protocol profile are correctly represented. QuantumTrustKG does not re-prove ML-KEM, hybrid TLS, password authenticated key exchange (PAKE) combiners, oblivious pseudorandom functions (OPRFs), or signature schemes. §10 states the consequences of that boundary.

Definition 1 (Admissibility). $p \in A_{\text{dir}}(e, t)$ iff (a) both endpoints of e support p , (b) p passes every hard gate active on e (policy class $\text{Security}(p) \succeq \text{Req}(e)$, deprecation and proof status, threat advisory), (c) every fact used to evaluate (a)–(b) is fresh and non-contradictory at t , and (d) p is rollout-feasible, i.e. promotable without forcing a peer into an unsupported mode.

When $A_{\text{dir}}(e, t) = \emptyset$ and active policy explicitly authorizes fallback, define $A_{\text{fb}}(e, t)$ by replacing only the direct class threshold with the policy’s effective fallback requirement. Endpoint support, freshness, conflict, deprecation, advisory, proof-status, and rollout gates are unchanged. Fallback is unavailable in every other case.

Theorem 1 (Safety: admissibility and class preservation. G1–G2). *For every edge e that QTPO promotes ($\pi(e) \in \mathcal{P}$), either $\pi(e) \in A_{\text{dir}}(e, t)$ and $\text{Security}(\pi(e)) \succeq \text{Req}(e)$, unless $A_{\text{dir}}(e, t) = \emptyset$ and active policy explicitly permits hybrid fallback, in which case $\pi(e) \in A_{\text{fb}}(e, t)$ is a hybrid profile.*

Proof. Algorithm 2 returns a profile only from A , which holds either direct candidates or, when none exists and policy permits it, fallback candidates. Consider the two cases.

Main path. $\pi(e)$ is chosen as $\arg \min_{p \in A} \text{Score}(p, e)$ where $A = \{p \in C_e \mid \text{FILTERGATES}(p, e, t)\}$. Membership in C_e gives Def. 1(a), since C_e contains only profiles both endpoints advertise support for. FILTERGATES returns true only when gate (i) (class), gates (ii)–(iii) (deprecation, proof status, advisory), gate (iv) (freshness and non-contradiction of every fact read while evaluating (a)–(b)), and gate (v) (rollout feasibility) all hold. These are exactly Def. 1(b)–(d). Hence $A \subseteq A_{\text{dir}}(e, t)$ and $\pi(e) \in A_{\text{dir}}(e, t)$. Gate (i) is precisely $\text{Security}(p) \succeq \text{Req}(e)$, which gives the class claim. Since $\arg \min$ selects an element of A , the ranking function cannot affect either conclusion.

Fallback path. Reached only when no direct candidate survives and POLICYPERMITSHYBRID(e) holds. The returned h is drawn from

$$H = \{h \in C_e \mid h \text{ hybrid and robust, FILTERGATES}_{\text{fb}}(h, e, t)\},$$

where $\text{FILTERGATES}_{\text{fb}}$ evaluates the gates under the hybrid-fallback policy. h satisfies Def. 1(a) and (c)–(d) unchanged, and satisfies (b) with the class gate evaluated against the relaxed requirement that the matching policy explicitly authorizes. This is the stated exception, and it is entered only when the direct admissible set is empty. Thus fallback cannot bypass any non-class gate.

No other return statement yields a profile: every remaining exit returns blocked. The readiness check only delays the commit of p^* and never changes it. $\square$

Theorem 2 (Fail-closed promotion. G4). *If any fact required to evaluate Def. 1 for e is stale, contradictory, unavailable, or mesh-inconsistent at t , then QTPO performs no new promotion on e : it preserves the last committed assignment and records `promotionBlocked` with the failed gate as the reason. It does not establish traffic denial or continuous admissibility of the retained profile.*

Proof. We case-split on how the fact fails.

Stale or contradictory. Gate (iv) of FILTERGATES evaluates $\neg \text{fresh}(f, t) \vee \text{contradicted}(f)$ over every fact $f \in \text{facts}(p, e)$ read while checking p , and returns false for that p . Every candidate whose justification touches the offending fact

is therefore removed from A (and, under the fallback branch, from H). If this empties both sets, the algorithm reaches the final **return blocked**. Removal happens *before* ranking. A removed candidate cannot be reinstated by a cost comparison.

Unavailable. If the graph store is unreachable, the reconciliation loop of Algorithm 1 short-circuits the edge before QTPO is invoked: it preserves $\pi(e)$ and marks the edge blocked with reason **graph-unavailable**. No candidate set is computed. No promotion can be emitted.

Mesh-inconsistent. A profile that clears all five gates is still committed only if $\text{MESHREADY}(e, p^*)$ holds, i.e. both Istio objects exist and both workloads report ready. When it does not, the algorithm returns **blocked** and the previously committed profile stays active. The commit step of Algorithm 1 records the edge as **Applying** with that previous profile and retries. A partially applied change is never recorded as active.

In all three cases the output is either the preserved previous assignment or **blocked**, never a profile of lower class than the previous one. Since **blocked** carries the first failed gate as its reason, the outcome is also auditable. $\square$

Proposition 1 (Hybrid robustness inheritance. G3). *If policy permits hybrid fallback and the selected hybrid profile is modelled (by its standardization or top-venue source) as secure when at least one of its component assumptions remains hard, then the assignment inherits that one-of-many robustness, and the recorded profile metadata continues to identify the components.*

Proof. QTPO considers a hybrid profile only on the fallback branch, that is, only when no pure quantum-safe profile is admissible and policy explicitly authorizes the transition. It selects among registered profiles and never constructs or modifies a combiner. The combiner’s components—and hence the source’s robustness claim—are unchanged by selection. Materialization copies the profile identifier and its declared class into the emitted **PeerAuthentication** and **Destination Rule** annotations without rewriting them. The property is inherited from the source rather than re-proved here. $\square$

Proposition 2 (Weight-independence, complexity, off-path latency).

(i) $A_{\text{dir}}(e, t)$ and $A_{\text{fb}}(e, t)$, and hence Theorems 1–2, is independent of the score weights and of the hysteresis δ , weights only reorder candidates within the applicable admissible set. (ii) One reconciliation evaluates selection in $O(\sum_{e \in E} |C_e| + Q(G))$, and an incremental one in $O(\sum_{e \in E'} |C_e| + Q(G))$ over the affected set $E' \subseteq E$. (iii) Selection runs only on watch events or scheduled evidence expiry. It adds no per-request data-path latency. It is idempotent on an unchanged graph, and δ prevents oscillation under bounded churn.

Proof. (i) A is constructed by **FILTERGATES**, whose five predicates read the edge, the candidate profile, the policy set, advisories, provenance facts, and rollout state. None of them reads w_o, w_r, w_m, w_h or δ . The weights appear only inside **Score**, which is applied to A after it is fixed, and δ only in the switch decision between two members of A . Hence A is a function of the graph state alone,

and both theorems—which quantify over members of A —are unaffected by any reweighting.

(ii) For one edge, building C_e costs one indexed lookup per endpoint capability set, `FILTERGATES` is a single pass over C_e with $O(1)$ work per gate given the graph query results, and `arg min` is linear in $|A| \leq |C_e|$. Summing over edges and adding the graph-query cost $Q(G)$ gives the stated bound. A naive implementation that checked every profile against every edge would instead cost $O(|E| |\mathcal{P}|)$. Because a service, policy, or advisory change triggers recomputation only for incident or matched edges—unaffected edges retain their capabilities, requirements, threat filters, and rollout state—the incremental bound follows by restricting the sum to E' .

(iii) The controller subscribes to watch events on Services, the three input CRDs, and mesh objects. No code path invokes selection from the request path. A promotion changes the data plane only through Istio configuration already in force. Idempotence holds because the gates and `arg min` are pure functions of graph state. An unchanged graph yields the same $\pi(e)$ and no write. Finally, a switch away from the active profile requires a score gap exceeding δ . An oscillation of amplitude below δ cannot produce repeated promotions. $\square$

Mechanization. We mechanize Theorems 1–2 and Prop. 1 in Coq/Rocq 9.1.1 under `formal/coq`. The modules `Model`, `QTP0`, `Guarantees`, and `Examples` define an abstract typed control-plane state with services, edges, policies, protocol profiles, provenance, rollout state, graph availability, and mesh readiness. The checked theorem names are `G1_promoted_admissible` (Thm. 1, `admissibility`), `G2_security_class_preserved` (Thm. 1, `class`), `G3_hybrid_metadata_preserved` (Prop. 1), and `G4_fail_closed` (Thm. 2). Immediately before committing a promotion, the Go controller re-checks it with an independent admissibility checker that mirrors the Coq predicates `direct_candidate` and `fallback_candidate` field-for-field (endpoint support, threat status, class, hybrid robustness and policy permission, graph availability, provenance, rollout feasibility, mesh readiness). A differential test compares both the selector and the checker with an executable transliteration of the Coq `select` on generated cases. The in-cluster measurements of Sec. 8 predate this controller: the release they were recorded with kept the original candidates when the two endpoints advertised disjoint profile sets, recorded a profile as active once its Istio objects were applied rather than after both workloads were ready, and did not read deprecation, proof-status or advisory metadata at run time. For that release, therefore, the endpoint condition of Thm. 1 and the readiness condition of Thm. 2 held only where those cases did not arise.

Both theorems follow from the filter order and promotion rule. The proofs above are case analyses over the two returning branches of Algorithm 2. These are *control-plane* guarantees over declared metadata rather than a wire-level proof of the bytes on the connection. A compromised workload could advertise quantum-safe capability, negotiate classical TLS, and leave both theorems intact. We treat RDF, SPARQL, and SHACL as implementation encodings rather than as part of the model. We do not prove primitive-level cryptographic security, and

we do *not* claim a machine-checked refinement from the Coq development to the Go controller. The field-for-field checker and its differential test are engineering evidence of correspondence rather than a proof of it.

7 Implementation

Our prototype is a Go controller (built on `controller-runtime`, Go 1.22) that stores the Quantum Trust Graph in Apache Jena Fuseki and materializes decisions as Istio objects, which enforce ordinary mTLS and carry the selected profile as control-plane metadata. It assigns a profile per edge, gates each promotion on endpoint capability, provenance, policy and threat metadata, records the profile as active only after both workloads are ready, and reports an explicit blocked status otherwise.

Custom resources. Four custom resource definitions (CRDs) hold the control-plane state. Three are operator inputs and one is controller-owned. A `QuantumSecurityPolicy` states the required class and trust boundary for a set of edges. A `CryptoCapabilityProfile` states which profiles a workload supports, together with a source and a freshness window (300s by default). A `MigrationPlan` drives staged rollout. The controller-owned `AssignmentStatus` records, for each edge, the desired and active profile, the assignment state, and the reason for it.

Reconciliation pipeline. On each watch event the controller re-runs only the touched edges. It also schedules a reconcile at the earliest evidence-expiry deadline. The pipeline has five stages. *Sync* writes the current cluster state to Fuseki. *Query* reads each edge’s candidate profiles, their threat metadata, and active policies with SPARQL, checking freshness and contradictions, and intersects the candidates with both endpoints’ fresh capability records. *Select* runs QTPO (Sec. 5). *Synthesize* emits, for each promoted edge, a `PeerAuthentication` and a `DestinationRule` whose `profile` annotation is the property a migration changes (Fig. 2). *Commit* applies these objects, keeps the edge `Applying` with its previous profile while the objects or either workload are not `Ready` (re-queuing it), and records the new profile as active only once both are ready and the independent checker of Sec. 6 agrees. Unsupported endpoints, stale facts, policy conflicts, and graph outages all become auditable promotion-blocked states. None becomes a new assignment or a traffic-denial rule.

Consistency under stale reads. Kubernetes rather than Fuseki, is the source of record: Services, the three input CRDs, and the controller-owned `AssignmentStatus` objects hold authoritative state, and Fuseki is a derived query cache rebuilt from them on each sync. Promotion additionally requires the fact to pass the freshness gate ($t \leq \text{observedAt} + \text{freshnessWindow}$, 300s by default). A read older than that window blocks-safe rather than promoting, which bounds stale-read exposure to the window. Under a flapping endpoint—Fuseki repeatedly reachable and unreachable, or capability claims oscillating—two mechanisms damp the oscillation: reconciliation is idempotent on an unchanged graph, and score hysteresis ($\delta=0.05$) keeps the current assignment unless a competitor is

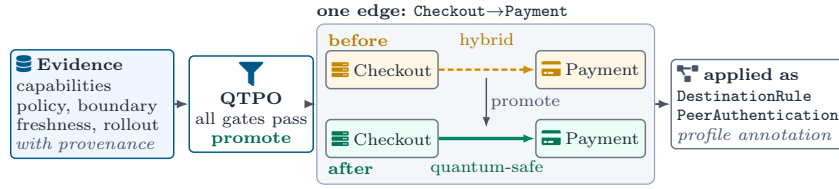


Fig. 2. What a migration does to one edge. When all gates pass, QTPO promotes `Checkout→Payment` from hybrid to quantum-safe TLS, and the mesh objects carry the new profile.

materially better. Reconciliation keys are re-enqueued rather than dropped. A promotion deferred during a flap is retried once fresh, consistent facts return. The controller also snapshots a monotonic evidence-cache revision before selection and rechecks it before commit. A concurrent policy or capability update blocks the promotion and forces a fresh pass. This does not provide a transactional snapshot across Fuseki and the Kubernetes API server, which remains a residual consistency limitation.

Integrity and recovery. We assume that capability publishers, policy authors, and the status writer are distinct RBAC subjects. That separation is the sole integrity boundary on the annotation layer (§10). Leader election serializes reconciliation. Generation-aware status writes drive recovery: after a crash, the next reconcile re-checks admissibility and either confirms or rolls back. Because new promotions require the graph, a Fuseki outage blocks them while the controller preserves the last committed assignments. On recovery it resynchronizes from Services, CRDs, and status objects before replaying pending reconciliations.

Capability provenance in the current prototype is operational rather than cryptographic. The controller records source, observation time, freshness, and contradiction notes, but it does not verify signed workload claims or bind them to identities issued by the Secure Production Identity Framework for Everyone and its runtime (SPIFFE/SPIRE). Binding claims to workload identity, to `cert-manager/SDS` (Secret Discovery Service) material, to signed capability attestations, and to dual classical/post-quantum certificate chains is future hardening rather than a property of the current artifact.

Portability. The contract is mesh-agnostic. Linkerd, Consul, or Cilium would reuse the same evidence inputs, gate semantics, and blocked-status vocabulary, changing only the objects that are materialized at the end of the pipeline.

8 Evaluation

We study controller safety and competing selectors (RQ1), behavior on sampled production service graphs (RQ2), migration cost (RQ3), and orchestration latency and semantic-core scalability (RQ4).

Throughout, every reported quantity is marked [M] when it is directly measured and [P] when it is a modelled parameter or an output of a cost model. Section 8.6 explains why that distinction matters for RQ3, the only research question for which a headline figure includes modelled values.

8.1 Experimental Setup

Harnesses. The four harnesses have distinct scopes.

(H1) *Archived in-cluster earlier-release replay.* The full Go controller, Fuseki, and Istio on 50/100/200-service testbeds. It supplies the end-to-end evidence for RQ1 compliance (Table 3), RQ4 latency (Table 11), and the resource footprint. Thirty repetitions per scale. Wilson 95% intervals on the per-edge compliance proportion. Median and p95 latency. The H1 values are re-emitted from a recorded run: the artifact contains neither the raw per-repetition logs nor an automated in-cluster driver. They are replayed rather than re-measured. That run used an earlier controller release which kept the original candidates when the two endpoints advertised disjoint profile sets, recorded a profile as active once its Istio objects were applied rather than after both workloads were ready, and did not read threat metadata at run time. For it, Theorems 1–2 held only where those cases did not arise (Sec. 6). The released controller closes all three paths, and H1 has not been repeated with it. H1 is historical integration evidence only and is not used to validate the released controller’s guarantees.

(H2) *Cluster-free selection harness.* A deterministic harness (`make baseline-compare`) that isolates the *selection logic* from Kubernetes, Fuseki, and Istio so that every baseline can be run against an identical edge population. It is the only harness in which competing systems appear—service-level crypto-agility (SL-CA), opportunistic negotiation (OPP), Enterprise-Level Crypto-Agility (ELCA) [34], automated post-quantum TLS configuration profiling (ConfigProfile) [35], a fixed mesh-wide hybrid posture (Static), and the NoKG/NoProv ablation—and its meshes are constructed to be harder than the H1 testbeds (see below). Seed 20260509, 30 repetitions per scale. H2 calls the controller’s selection function directly on candidate lists that are already restricted to the profiles both endpoints support, and applies no mesh objects. The endpoint-capability and readiness changes of the released controller do not affect it. Its outputs are byte-identical before and after them.

(H3) *Semantic-core stress.* The deterministic parse-and-filter reasoning core alone (Go fixture path), with Fuseki, Kubernetes watches, and Istio churn *excluded*. It supports the 500/1000/2000-service rows (Table 12) and nothing else. Seed 20260509, median of 15 iterations.

(H4) *Native TLS runners.* OpenSSL handshake timing and end-to-end negotiated-group validation (Table 10). These are real transport measurements on a stock host rather than in-mesh Envoy negotiations.

The full-controller scale and churn harness additionally reports rows only when a `kind` (Kubernetes-in-Docker) environment is present. On our host it was not. Those rows are recorded as unavailable rather than extrapolated (Table A2), and the grounded end-to-end claim stays the 50–200-service H1 measurement.

Table 1. Experimental configuration behind every number in Sec. 8.

Harness	Software	Repetitions	Scope / exclusions
H1 in-cluster	Go 1.22 controller (<code>controller-runtime</code> , earlier release), Apache Jena Fuseki, Istio	30 reps per scale; seed 20260509	Full path: watches, SPARQL, Istio objects, status writeback; values replayed
H2 cluster-free	Same Go selection package, no cluster	30 reps per scale; seed 20260509	Selection logic only; excludes Fuseki, watches, Istio
H3 semantic core	Go fixture parser/filter	median of 15 iterations; seed 20260509	Parse + filter only; excludes Fuseki, watches, Istio churn
H4 native TLS	OpenSSL 3.6.2 (7 Apr 2026), default provider	300 handshakes, 30 wire trials	Stock host rather than in-mesh Envoy
Data model	rdflib 7.6.0, SQLite 3.53.2, networkx 3.6.1	median of 5 repeats	Same admissibility join under three encodings
Proofs	Coq/Rocq 9.1.1	—	Abstract control-plane model

Workloads. H1 and H2 use generated meshes with an edge/service ratio of about two. In H2, the meshes include requirement divergence at shared destinations, the pattern that motivates the paper: a regulated edge and a hybrid-capped partner edge enter one destination service. About 3% of edges carry stale or contradictory provenance. We also run H2 over *real* service-dependency graphs extracted from the Alibaba microservices trace [36] (Sec. 8.5), and over the Online Boutique service graph [26].

Software and determinism. Table 1 records the versions, seeds, and repetition counts behind every number in this section. H2, H3 (counts), H4 and the weight sweep are regenerated by the artifact and are deterministic given the stated seed (H3 and H4 timings are host-dependent). The per-repetition sub-seeds are derived from it. A re-run reproduces those CSVs byte for byte. The H1 numbers and the modelled setup-cost figures are instead re-emitted from a recorded canonical file by the artifact’s `paper-results` command, which replays them and does not re-measure them. Appendix A maps each claim to the artifact command and output file that produces it.

8.2 Metrics

Let E be the set of managed communication edges in a run. Every metric below is computed over *all* managed edges, and every one is computed from *declared* labels: the capability claims workloads publish, the policy resources operators write, and the security class recorded on each registered protocol profile.

Declared-label policy compliance.

$$C_{\text{decl}}(E) = \frac{1}{|E|} \sum_{e \in E} \mathbf{1}_{\text{compliant}}(e) \times 100,$$

where $\mathbf{1}_{\text{compliant}}(e) = 1$ only when the controller promotes a profile that satisfies endpoint support, the derived security requirement $\text{Req}(e)$, proof-status and

Table 2. The two declared-label policy compliance numbers (200 services), and why they differ.

Symbol	Harness	C_{decl}	Blocked	Unsafe	Why it differs
$C_{\text{decl}}^{\text{cluster}}$	H1 in-cluster, 200 svc	99.1%	0.9%	0.0%	Testbed edge mix; few edges must be refused
$C_{\text{decl}}^{\text{harness}}$	H2 cluster-free, 200 svc	96.5%	3.4%	0.0%	Divergent edges at shared destinations, $\approx 3\%$ corrupted facts by construction

family restrictions, provenance freshness, and rollout readiness—or a hybrid profile where active policy explicitly permits fallback.

The qualifier *declared-label* matters. The requirement $\text{Req}(e)$ and the class $\text{Security}(p)$ are both read from the same metadata that the decision consumes. A workload that advertises `quantum_safe` support and then negotiates classical TLS is counted as compliant. C_{decl} therefore measures whether the control plane decides correctly *given its inputs*. It is not a measurement of the cryptography on the wire. We retain this qualifier in the abstract, tables, figures, and conclusion.

Invalid promotion rate $U(E)$ is the share of managed edges promoted despite failure of the complete independently evaluated declared-input predicate: endpoint compatibility, policy class, family/proof status, advisory, fresh noncontradictory facts, or rollout readiness. It does not establish that capability declarations match wire behavior.

No-promotion rate $B(E)$ is the share of edges the controller declined to promote. We additionally record whether a previous active profile is retained. Neither outcome means traffic is denied. A non-promotion contributes 0 to C_{decl} , which makes C_{decl} deliberately conservative: $C_{\text{decl}} + B + U \approx 100$, up to the gap between the per-repetition median of C_{decl} and the pooled rates.

Two compliance numbers, two workloads. C_{decl} is reported on two different workloads, and the two must not be compared directly. We therefore label them distinctly everywhere:

- $C_{\text{decl}}^{\text{cluster}}$ — archived from the earlier-release H1 in-cluster testbeds (Fig. 3(a), Table 3). At 200 services it is 99.1%, with 0.9% not promoted and 0% invalid under the historical predicate.
- $C_{\text{decl}}^{\text{harness}}$ — measured on the H2 cluster-free harness (Table 4). At 200 services it is 96.5%, with 3.4% not promoted and 0% invalid.

Both are all-edge rates computed by the same formula. The 2.6-point difference is the no-promotion share. H1 and H2 use different instrumentation and do not jointly validate the current controller. H2 contains divergent edges at shared destinations and 3% corrupted facts. The same controller correctly refuses more edges. Table 2 summarizes the mapping between each value and its workload.

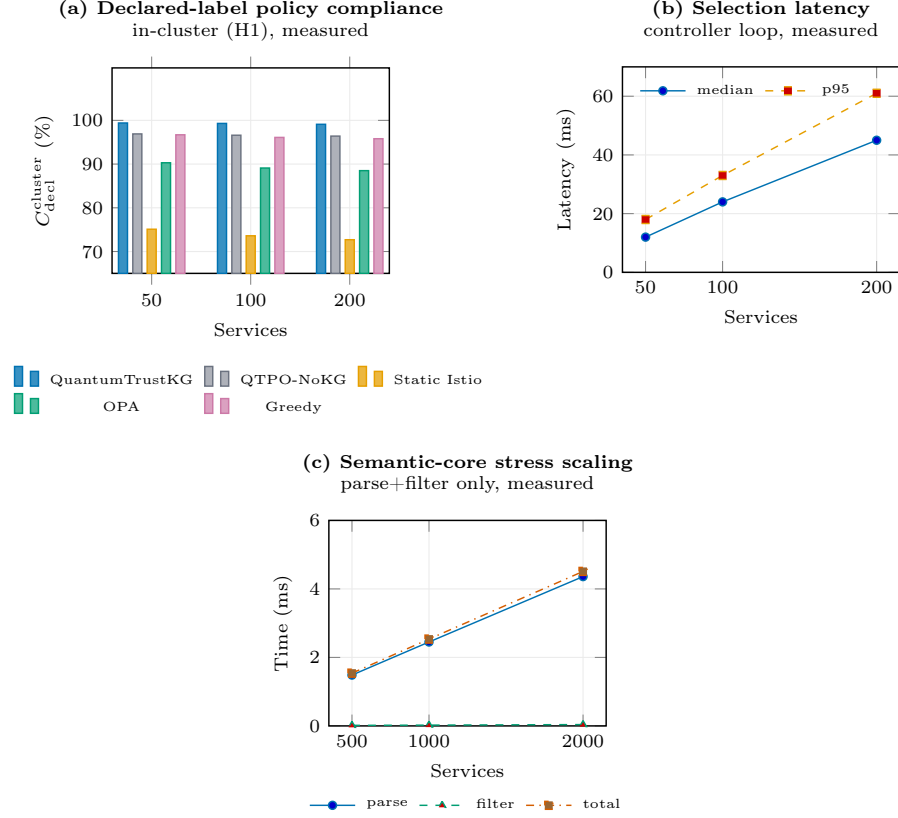


Fig. 3. Evaluation trends [M]: (a) in-cluster declared-label policy compliance $C_{\text{decl}}^{\text{cluster}}$, (b) selection latency, (c) semantic-core stress scaling.

8.3 RQ1: Safe Edge-Level Decisions

Archived in-cluster compliance (H1). In the earlier-release replay, $C_{\text{decl}}^{\text{cluster}}$ falls from 99.4% [99.1, 99.6] at 50 services to 99.3% [99.0, 99.5] at 100 and 99.1% [98.9, 99.3] at 200, against the NoKG/NoProv ablation’s 96.9% [96.5, 97.3], 96.6% [96.2, 96.9] and 96.4% [96.0, 96.7]. Wilson intervals separate them at every scale. The archived instrumentation recorded zero invalid promotions under its historical predicate, and residual edges were not promoted with an actionable status reason (Fig. 3(a), Table 3, brackets are Wilson 95% intervals on the per-edge compliance proportion over 30 repetitions). The other in-cluster baselines—Static Istio, Open Policy Agent (OPA)-guided enforcement, operator-driven Manual rollout, and a Greedy non-graph selector—decline as edge diversity grows. The Online Boutique fixture has only 11 managed edges per run. Its interval is correspondingly wider. NoKG/NoProv was not run in-cluster on that fixture, and its ablation there appears in Sec. 8.8. Median selection latency grows from 12 ms at 50 services to

Table 3. In-cluster declared-label policy compliance $C_{\text{decl}}^{\text{cluster}}$ (%) by scale and baseline (H1, Wilson 95% intervals in brackets) [M].

Scale	QuantumTrustKG	NoKG/NoProv	Static Istio	OPA	Manual	Greedy
50 services	99.4 [99.1, 99.6]	96.9 [96.5, 97.3]	75.1	90.3	92.4	96.7
100 services	99.3 [99.0, 99.5]	96.6 [96.2, 96.9]	73.6	89.1	91.8	96.1
200 services	99.1 [98.9, 99.3]	96.4 [96.0, 96.7]	72.7	88.5	91.3	95.8
Online Boutique (11 edges)	99.3 [97.9, 99.8]	—	76.4	90.7	92.9	96.4

45 ms at 200 [M]—a control-loop cost, since QTPO runs on configuration change, never per request (Sec. 8.7).

Controlled baseline comparison (H2). Since no prior post-quantum-migration system ships as a runnable per-edge control plane, we distil the dominant literature patterns into reproducible selectors and run them against an identical edge population (Table 4). The baselines are: Enterprise-Level Crypto-Agility (ELCA) [34], which drives cryptography from centralized per-service policy, automated post-quantum TLS *configuration profiling* (ConfigProfile) [35], a per-endpoint crypto inventory with provenance, validated by its authors on 8,443 real Nginx configurations, service-level crypto-agility (SL-CA), the per-service or per-namespace posture pattern, opportunistic negotiation (OPP), which takes the strongest mutually supported profile per edge but trusts capability claims at face value, Static, a fixed mesh-wide hybrid posture, and NoKG/NoProv, the combined ablation. Section 8.4 states exactly how faithful these re-creations are. In Table 4, $C_{\text{decl}}^{\text{harness}}$ counts edges promoted at or above the required class (or to a policy-permitted hybrid fallback) and Unsafe counts edges promoted despite any failed admissibility gate. $C_{\text{decl}}^{\text{harness}}$ is a per-repetition median and Invalid a pooled rate, and the 50- and 100-service meshes agree with the 200-service ones within 1.2 points.

Only QTPO has zero unsafe promotions, at every scale. The comparison also separates the two ingredients QuantumTrustKG combines. *Per-edge granularity* costs ELCA, SL-CA, and ConfigProfile 11.2–13.7% unsafe promotions, because each applies one posture per service or endpoint and a service whose inbound edges diverge has no single correct posture, *provenance gating* lowers ConfigProfile’s to 11.2%, below ELCA’s 13.5%, while ELCA, SL-CA, OPP, and NoKG/NoProv all promote on facts QTPO blocks. Only QTPO has both.

The H2 comparison starts without active assignments. Thus the reported no-promotion counts are separate from retained-active outcomes, whose count is zero for every H2 row. Controller tests separately exercise retention of a previously committed profile after a blocked promotion.

8.4 Scope of the Re-Created Baselines

Every competing system in Table 4 is our re-creation rather than an execution of the authors’ original code. The reported gaps must be interpreted within that constraint.

Table 4. Controlled, cluster-free selection comparison (H2, 200-service meshes, 30 reps, seed 20260509) [M]. SL-CA = service-level crypto-agility. OPP = opportunistic negotiation.

Method	$C_{\text{decl}}^{\text{harness}}$ (%)	Invalid (%)	Invalid edges	No promotion / cause
QTPO (ours)	96.5	0.0	0	403 / complete gating
NoKG/NoProv	96.5	2.5	303	100 / no provenance gating
OPP (opportunistic)	96.5	3.4	403	0 / no provenance gating
ConfigProfile [35]	85.7	11.2	1336	304 / per-endpoint
Static (fixed hybrid)	86.7	13.1	1557	0 / no per-edge requirement
ELCA [34]	86.1	13.5	1614	0 / central policy
SL-CA (service-level)	85.7	13.7	1640	0 / service-level

All rows: 11,929 pooled edge-repetition observations over 30 repetitions.

Implementation basis. For each baseline we reimplemented only the *selection rule*—one function in the H2 harness—and left everything else identical across methods: the same edge population, the same requirement model, the same profile registry, the same repetitions and seed. Table 5 records the source each rule was derived from and what our version omits. Two of the four are published systems with no public implementation we could obtain (ELCA is an IACR ePrint report, ConfigProfile is described in an arXiv preprint), and two (SL-CA, OPP) are *patterns* rather than artifacts—the per-service posture that policy-as-code and per-service mesh configuration typically produce, and the “negotiate the strongest mutually supported group” behaviour of TLS—so for those two there is no original code that could have been run.

Sources of variation. Our versions reproduce each system’s *decision granularity* and its *treatment of provenance*, because those are the two axes the comparison is about. They omit everything else: ELCA’s central crypto service, key management, and monitoring and audit pipeline, ConfigProfile’s real configuration-scanning pipeline and its inventory store, and any tuning its authors would apply. A re-creation can therefore be wrong in either direction. It may flatter a baseline by giving it our clean requirement model, or penalize it by omitting an engineering mitigation its authors would have deployed. We did not attempt to estimate the size of that error. The absolute gaps in Table 4 therefore should not be interpreted as estimates for the original systems.

Controlled ablation. NoKG/NoProv is our controller with a coupled set of functions removed—the graph-backed model and the provenance locality, freshness, contradiction detection, and decision traces that depend on it. It runs the same code path, the same filters, the same weights, and the same hysteresis. Its gap to QTPO is 2.5% unsafe promotions against 0.0% on the H2 meshes, 3.19% against 0.00% on the real 200-service topology (Table 6), and 96.4% against 99.1% $C_{\text{decl}}^{\text{cluster}}$ in-cluster (Table 3)—about 3% that survives every other part of the design. The controlled result is therefore that the combined graph-backed representation and provenance gate eliminates an invalid-promotion rate of about 3% (2.5–3.6% across the H2 meshes, the sampled topologies, and Online Boutique). It does not isolate RDF representation as the cause. The larger 11–14% gaps against ELCA, ConfigProfile, and SL-CA measure a *granularity* limitation

Table 5. Faithfulness of the re-created baselines: source, what we reimplemented, and what our version omits.

Baseline	Source	What we reimplemented	What our version omits
ELCA [34]	IACR ePrint 2023/1539; no public implementation obtained	Centralized per-service mandate: the strongest class any of a service's edges requires, applied uniformly; no per-edge provenance gate	Central crypto service, key management, monitoring/audit pipeline, the authors' policy language and tuning
ConfigProfile [35]	arXiv preprint, no public implementation obtained	Per-endpoint configuration ceiling with provenance tracking that fails closed on stale facts but cannot express per-edge divergence	The configuration-scanning pipeline over real deployments, the inventory store, deployment automation
SL-CA	Per-service / per-namespace posture pattern	Strongest class supported on <i>all</i> of a service's inbound edges, applied uniformly, with no provenance gate	Nothing specific: there is no reference implementation to diverge from
OPP	TLS-style best-available negotiation pattern	Per-edge strongest mutually supported profile, capability claims trusted at face value	Nothing specific: there is no reference implementation to diverge from
NoKG/NoProv	Our combined ablation	Same ranking path; graph-backed join, provenance locality, freshness, contradiction detection, and traces removed	Does not isolate representation from provenance gating

that we argue is intrinsic to a per-service posture (Sec. 8.5 gives the mechanism), but they are measured against our reconstruction of those systems rather than against those systems.

8.5 RQ2: Real Production Service Graphs

We next apply H2 to service-dependency graphs sampled from a production trace. SL-CA and ConfigProfile increase from 11–14% unsafe on the synthetic meshes to 37–60% on these samples.

Data. We extract real caller-to-callee edges from one subset of the Alibaba microservices trace [36]: a single 3-minute call-graph bucket (13,332,356 call rows), yielding 16,247 services and 48,746 unique edges after removing self-loops, with a heavy-tailed degree distribution (median 3, maximum 2971). We then take connected breadth-first subgraphs from the highest-degree hub at six scales. *Only the topology is real:* the trace carries no cryptographic policy. Per-edge requirements and supported profiles are assigned by the same calibrated model used for the synthetic meshes. The comparison therefore varies topology while retaining the same requirement model.

Results by system size. Table 6 gives the full breakdown.

QTPO has 0.00% invalid promotions at all six scales [M] (Fig. 4(a), log-scale size, grey bars = share of edges into the single top hub), from 50 to 3,692 unique

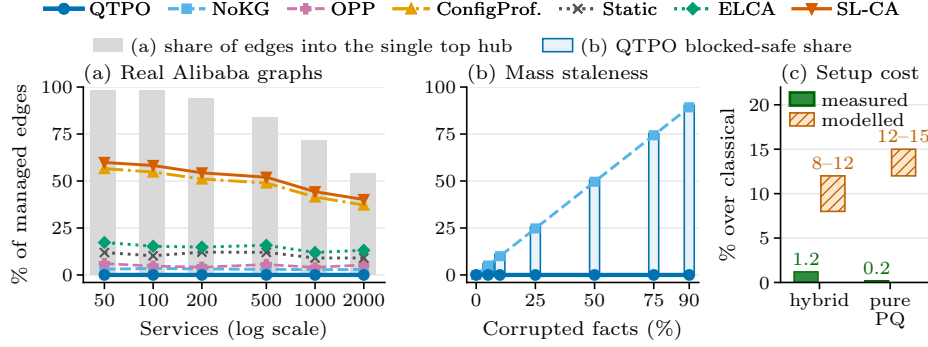


Fig. 4. (a,b) Unsafe promotions (lines, % of managed edges, 30 reps) [M]: (a) real Alibaba subgraphs, (b) mass staleness, 200 services. (c) Setup cost over classical: measured (solid, [M]) vs. modelled (hatched, [P]). PQ = post-quantum.

Table 6. Unsafe promotion rate (%) on *real* Alibaba service topologies by system size (H2 over real edge lists, 30 reps, seed 20260509) [M].

Services	Pooled obs.	QTPO	NoKG/NoProv	OPP	ELCA	Static	ConfigProf.	SL-CA
50	1,500	0.00	3.07	6.07	17.27	11.93	56.60	59.87
100	3,030	0.00	3.47	4.72	15.28	10.20	54.75	58.25
200	6,360	0.00	3.19	4.14	14.81	12.15	51.07	54.34
500	17,850	0.00	3.01	5.56	15.80	12.03	48.94	52.03
1000	41,940	0.00	2.79	4.03	11.96	9.02	41.48	44.30
2000	110,760	0.00	2.89	5.34	13.13	9.04	37.19	40.15

Structure of the same topologies, computed from the edge lists: share of edges terminating at the single highest-fan-in service — 98.0% (50 svc), 98.0% (100), 93.9% (200), 83.9% (500), 71.5% (1000), 54.1% (2000). In the full 16,247-service trace, 6.1%. Taking the top 5% of services instead: 100/100/98.1/92.8/83.5/74.6%, versus 52.6% in the full trace.

edges per repetition. The table’s 1,500–110,760 counts are pooled edge-repetition observations over 30 repetitions. A per-edge gate does not generalize across a service’s incoming edges and is therefore unaffected by hub concentration in this metric.

SL-CA and ConfigProfile have unsafe-promotion rates about four to five times those on synthetic meshes of the same size. At 200 services SL-CA rises from 13.7% unsafe (synthetic, Table 4) to 54.3% (real), and ConfigProfile from 11.2% to 51.1%. ELCA rises more modestly, from 13.5% to 14.8%, because its failure mode is a different one: it over-mandates a central class onto edges that cannot support it, rather than under-protecting a divergent edge.

The real-topology rates decrease monotonically with scale: from 59.87% (SL-CA) and 56.60% (ConfigProfile) at 50 services to 40.15% and 37.19% at 2000 [M], while NoKG/NoProv stays near 3% and the share of edges into the single top hub falls from 98.0% to 54.1%, against 6.1% in the full trace. The per-edge methods show no such trend.

Effect of hub concentration. The difference between the synthetic and sampled topologies is structural.

A service-uniform method must commit one class per destination service. SL-CA and ConfigProfile choose the strongest class that does not break *any* peer of that service, i.e. the minimum over the service’s inbound edges of the mutually supported class. That minimum is taken over a set whose size is the service’s fan-in. Synthetic meshes give each destination a fan-in of one to three. The minimum is taken over a handful of peers and is usually close to what each individual edge needs. Real service graphs are heavy-tailed: in our subgraphs a single hub absorbs most of the edges. The share of edges terminating at the top service is 98.0% at 50 services and still 54.1% at 2000 (see the note under Table 6). When one service receives almost every call, one legacy or partner peer anywhere in that fan-in drags the whole service’s posture down, and every other edge into that hub—including every regulated one—inherits the floor, producing the observed 37–60% range.

The same quantity explains the size trend. As the subgraph grows, the hub’s share of edges falls (98.0% $\rightarrow$ 54.1%). A larger fraction of edges terminate at smaller services whose fan-in is short and whose uniform posture is therefore closer to correct. The unsafe rate falls in step, from 59.87% to 40.15% for SL-CA. The rate is therefore associated with call-graph concentration rather than size alone. ELCA is affected differently because its rule takes a *maximum* over the service’s requirements rather than a minimum over its capabilities. Hub concentration causes it to over-mandate rather than under-protect. Its rate accordingly stays in the 12–17% band.

Sampling scope. Our subgraphs are breadth-first neighbourhoods of the highest-degree hub, which makes them *more* concentrated than the trace as a whole: the top service absorbs 54–98% of edges in the subgraphs, but only 6.1% in the full 16,247-service graph (taking the top 5% of services instead: 75–100% in the subgraphs against 52.6% in the full trace). The 37–60% band should therefore be read as the behaviour of a service-uniform posture in the hub-dominated part of a real system rather than as an average over a whole production mesh. The monotone trend in Table 6 is consistent with this: extrapolating it toward the full graph’s concentration would predict a lower, but still substantial, penalty. We report the measured points and do not extrapolate them.

Mass staleness. Under mass staleness, the provenance gate produces fail-closed behavior (Fig. 4(b), bars = QTPO’s blocked-safe share, Table 7): sweeping corrupted annotations from 0 to 90% over a 200-service mesh, QTPO holds zero unsafe promotions at every level [M] while blocked-safe tracks the injected fraction and C_{decl} degrades from 99.03% to 9.87%, whereas the provenance-blind ablation keeps the same throughput but turns each corrupted edge into an unsafe promotion, reaching 24.75/49.59/89.22% at 25/50/90%. Its unsafe rate tracks the injected corruption almost exactly. The real 200-service Alibaba topology shows the same zero [M]: there C_{decl} degrades from 100.00% to 9.91% and the ablation reaches 89.20% unsafe at 90% corruption. QTPO thus converts corrupted

Table 7. Fail-closed behaviour under mass staleness and adversarial annotations (200-service mesh, 30 reps, seed 20260509) [M].

Stale frac.	QTPO C_{decl} (%)	QTPO blk-safe (%)	QTPO unsafe (%)	NoKG unsafe (%)
0%	99.03	1.00	0.00	0.00
5%	94.16	5.97	0.00	4.99
10%	89.14	10.89	0.00	9.97
25%	74.43	25.57	0.00	24.75
75%	24.81	75.20	0.00	74.35
90%	9.87	90.13	0.00	89.22

evidence into blocked-safe outcomes, consistent with Thm. 2, whereas NoKG continues to promote.

8.6 RQ3: Measured and Modelled Migration Costs

This subsection separates measured handshake and rollout quantities from the modelled aggregate setup cost.

Setup-cost definition. Setup cost is a *relative* increase over a classical baseline for one migration mode m :

$$\text{SetupCost}(m) = \frac{\text{Setup}_m - \text{Setup}_{\text{base}}}{\text{Setup}_{\text{base}}} \times 100.$$

Here Setup is a dimensionless operator cost index rather than elapsed time or central-processing-unit (CPU) time alone, the archived calibration supplies its coefficients but not a measured conversion among unlike resources. It combines four normalized terms over a classical baseline: per-handshake CPU and wire cost (added on top of mTLS establishment), certificate and key material, staged rollout, and configuration churn in the mesh. Our calibrated model puts the aggregate at 8–12% for hybrid and 12–15% for pure post-quantum [P] (Table 9, Fig. 4(c), hatched).

Measured term. The handshake term is measured (Fig. 4(c), solid). On stock OpenSSL 3.6.2, which exposes ML-KEM natively, the median TLS 1.3 handshake over 300 handshakes is 8.746 ms with X25519, 8.850 ms with X25519MLKEM768 (+1.2%) and 8.759 ms with ML-KEM-768 (+0.2%) [M], consistent with earlier TLS benchmarks of lattice-based key exchange [45]. Thirty end-to-end client–server trials per profile all succeeded (30/30) [M] with the requested group read back from every connection, at wire p50 9.337 / 9.310 / 9.213 ms (Table 10). The post-quantum key exchange is therefore about a 1% effect on this host. The measurement uses OpenSSL rather than an Envoy sidecar, since stock Envoy/Istio does not negotiate post-quantum TLS on the wire.

Modelled terms. The other three terms—certificate and key material, the staged-rollout cost share, and configuration churn—are model parameters. None is separately measured, and they account for most of the 8–15%: if the measured component contributes about one percentage point, roughly 7–15 of those 8–15 points are modelled [P]. We therefore claim no aggregate migration cost here.

Table 8. Every input to the setup-cost calculation, with its provenance ([M] = directly measured, [P] = model parameter).

Input	Status	Value	Where it comes from
Handshake CPU/wire term	[M]	+1.2% hybrid, +0.2% pure PQ (median of 300 TLS 1.3 handshakes)	OpenSSL 3.6.2 native ML-KEM, Table 10
Wire-negotiation check	[M]	30/30 successful trials per profile; p50 9.337/9.310/9.213 ms	OpenSSL client-server, negotiated group read back
Certificate and key material	[P]	not separately measured	Model parameter; enters only the aggregate
Staged rollout cost share	[P]	not separately measured	Model parameter; the rollout <i>time</i> below is measured
Configuration churn	[P]	not separately measured	Model parameter; enters only the aggregate
Aggregate, hybrid	[P]	8–12%	Calibrated model output
Aggregate, pure PQ	[P]	12–15%	Calibrated model output
Perturbation envelope	[P]	6–15% hybrid; 9–19% pure PQ under $\pm 25\%$ on CPU/wire terms	Sensitivity run; compliance and latency unchanged
Calibration bias	[P]	+7–9% CPU, +4–5% wire (non-optimistic)	Recorded with the calibration; the estimate errs high
Rollout time	[M]	4–7 min hybrid; 8–12 min pure PQ	Controller logs
Rollout failure / rollback rate	[M]	0.0%/0.0% hybrid; 1.1%/0.6% pure PQ	Controller logs

Table 8 lists every input to the calculation, its value or status, and where that value comes from. We do not publish a per-term split because the artifact records the calibration as an aggregate range per mode rather than as four separately measured components, and inventing a split would present model structure as evidence.

Sensitivity and supporting evidence. The artifact provides three qualifications that bound the estimate without validating the aggregate. (i) *A perturbation envelope* [P]: perturbing the calibrated CPU and wire terms by $\pm 25\%$ moves the aggregate to 6–15% (hybrid) and 9–19% (pure post-quantum), and changes neither compliance nor controller latency—the cost model is an output of the system, never an input to a safety decision, which is why perturbing it cannot move a compliance number. (ii) *Calibration bias*: the calibration carries a positive bias of 7–9% on the CPU term and 4–5% on the wire term. The estimate errs high rather than low. (iii) *Measured rollout outcomes* [M]: rollout time, failure rate, and rollback rate in Table 9 are measured from controller logs—4–7 minutes and 0.0%/0.0% for hybrid, 8–12 minutes and 1.1%/0.6% for pure post-quantum—so the rollout *schedule* is grounded even where its *cost share* is modelled.

Interpretation of Fig. 4(c). The solid bar is the measured handshake term. The hatched bar is the modelled aggregate range. Any conclusion of the form “post-quantum migration costs 8–15%” rests on the hatched bar and inherits its status. The measured 0.2–1.2% handshake overhead indicates only that key exchange is not dominant under this specified aggregate-cost model.

Table 9. Setup cost [P] and rollout outcomes [M] (controller logs) by migration mode.

Mode	Setup cost (%) [P]	Rollout time (min) [M]	Fail rate (%) [M]	Rollback (%) [M]
Hybrid	8–12	4–7	0.0	0.0
Pure PQC	12–15	8–12	1.1	0.6

Table 10. Native TLS 1.3 measurements on stock OpenSSL 3.6.2 (native ML-KEM, default provider) [M].

Mode	Group	Negotiated	Handshake (ms)	Wire p50 (ms)	Overhead
Classical TLS 1.3	X25519	X25519	8.75	9.34	—
Hybrid ML-KEM	X25519MLKEM768	X25519MLKEM768	8.85	9.31	+1.2%
Pure PQ	MLKEM768	MLKEM768	8.76	9.21	+0.2%

Table 11. Selection latency, semantic sub-costs, and footprint (H1, 30 reps) [M].

Scale	Med. (ms)	p95	IQR	95% CI (med.)	Query	Rules	Ctrl.
50 services	12.0	18.0	4.0	11.4–12.7	4.1	2.8	12.0
100 services	24.0	33.0	7.0	22.9–25.2	9.3	5.9	24.0
200 services	45.0	61.0	11.0	43.1–46.8	18.6	11.7	45.0

8.7 RQ4: Orchestration Latency and Semantic-Core Scalability

Orchestration latency (H1).

Reconciliation stays off the data path (Fig. 3(b), Table 11). Median selection latency grows from 12 ms at 50 services to 45 ms at 200, with p95 up to 61 ms. QTPO runs after a policy, capability, threat, or migration change, never per request. This is a control-loop cost: it delays how quickly a new posture takes effect rather than how quickly a request completes. At 200 services the controller stays below 1 vCPU and 600 MB resident set size (RSS) in the archived run. In Table 11, all millisecond values are over 30 repetitions, the 95% confidence interval (CI) is a bootstrap interval on the median, IQR is the interquartile range, and Query and Rules are the semantic sub-times inside the controller loop.

Semantic-core scalability (H3).

Figure 3(c) and Table 12 isolate the semantic control plane: the deterministic parse-and-filter reasoning core only, run on generated Turtle fixtures through the Go parser/filter path, with Fuseki, Kubernetes watches, and Istio churn *excluded*. Read that way, the core scales near-linearly. At 500/1000/2000 services (6.4k/12.7k/25.4k triples) the parse-plus-filter total is 1.5/2.5/4.5 ms with 2.8/4.6/8.9 MB allocated, and zero unsafe profiles at every scale.

These rows are reproducibility stress evidence rather than a mesh-scale deployment measurement or end-to-end evidence. The full controller scale and churn harness emits timing only under a `kind` environment, which was unavailable on our host, rather than extrapolate, those rows are recorded as unavailable (Table A2), and the grounded end-to-end claim remains the 50–200-service H1 measurement above.

Table 12. Deterministic semantic-core stress (H3, excludes Fuseki, watches, and Istio churn) [M].

Scale	Edges	Triples	Parse (ms)	Filter (ms)	Total (ms)	Alloc. (MB)
500 services	1000	6.4k	1.484	0.016	1.540	2.832
1000 services	2000	12.7k	2.450	0.023	2.535	4.647
2000 services	4000	25.4k	4.360	0.039	4.511	8.883

Median of 15 iterations, seed 20260509. Promoted/blocked edges: 936/64, 1889/111, 3758/242. Zero unsafe profiles at every scale.

Table 13. Fault-injection evidence (controller fixtures) [M].

Scenario	Injected fault	Unsafe / blocked / pre- served	Recovery or resync	Status reason
Stale fact	Capability observation past its freshness window; 2 regulated edges	0 / 2 / 0	Fresh observation unblocks on next reconcile	stale-facts
Contradictory fact	Support claim conflicts with a newer advisory; 1 edge	0 / 1 / 0	Remove or supersede advisory, then replay	conflicting-facts
Graph outage	Fuseki ping/query fails at reconcile start; 3 edges	0 / 3 / 1	Resync from Services, CRDs, and status objects	graph-unavailable; graph-unavailable-preserved-active
Mesh mismatch	Staged batch reports unhealthy readiness; 2 edges in 1 rollout batch	0 / 2 / 1	Rollback phase freezes further batches	observed-unhealthy; rollback-triggered
Policy conflict	Strict PQ policy meets hybrid-only endpoints; 2 edges	0 / 2 / 1	Add a feasible profile or authorize explicit fallback	no-admissible-profile

8.8 Failure Injection and Additional Results

Failure injection. We inject five classes of fault, one class at a time, and check the controller’s response edge by edge (Table 13). Every case produces zero unsafe promotions: stale facts and contradictory advisories block the affected edges, a graph outage blocks new promotions while preserving the active assignment, an unhealthy staged batch freezes further batches and rolls back, and a strict post-quantum policy meeting hybrid-only endpoints blocks with **no-admissible-profile**. The injected faults exercise the controller and its fixtures. They are not measurements of a liboqs/Envoy post-quantum data path.

Blocked-safe outcomes. Because C_{decl} counts a blocked edge as non-compliant, the residual matters. Table 14 breaks the 200-service residual down by reason, as shares of the residual non-compliant edges rather than of all edges: unsupported endpoints and genuinely infeasible profiles account for 60% of it, stale and contradictory facts for a further 32%, and policy conflicts for 8%. Each category carries an operator-actionable status reason, and none of them is an accepted downgrade. Unsafe promotions are zero.

Table 14. Blocked-safe breakdown at 200 services (shares of the residual) [M].

Blocked-safe category	Share	Typical status reason	Operator remediation
Unsupported endpoints	29%	no-candidate-profiles	Upgrade endpoint capability or keep previous profile
Stale facts	18%	stale-facts	Refresh workload claim or provenance source
Contradictory facts/advisories	14%	conflicting-facts	Resolve source conflict before replay
No feasible profile	31%	no-admissible-profile	Add a compatible profile or split the migration plan
Policy conflict	8%	no-admissible-profile	Adjust policy or mark a temporary fallback explicitly
Unsafe promotions	0%	–	Not observed in any run

Weight sensitivity. On the 200-service H2 meshes (30 reps, seed 20260509, half of the edges carrying an existing active profile) we scale each weight and δ by 0.5 and 1.5 around the operating point, one at a time, and also set $\delta=0$. At every point $C_{\text{decl}}^{\text{harness}}$ stays at 96.47% and unsafe promotions at zero [M], while the promoted profile changes on up to 20.48% of promoted edges (for $w_o \times 0.5$ and $w_r \times 1.5$). This is the empirical companion to Prop. 2: the filter chain produces compliance and does not read the weights. The sweep only changes which admissible profile wins. Appendix E.2 gives the full sweep.

Online Boutique. On this recognizable microservice graph [26,37,38] (11 services, 30 reps) QuantumTrustKG reaches 99.3% median per-edge C_{decl} , 9 ms median and 21 ms p95 latency, and zero unsafe promotions, where the QTPO-NoKG ablation incurs 3.6% unsafe promotions (12 of 330 edges). The checked decision trace for **Checkout**→**Payment** is in Appendix C.2.

9 Case Study: Financial-Services Payment Processing

This case study illustrates the controller’s decisions in the payment-processing domain that motivated the design.

Context. We applied QuantumTrustKG to a payment-processing scenario of 85 microservices covering transaction authorization, fraud detection, settlement, and audit. The workload is *synthetic*, but shaped by common payment-platform structure: cardholder-data zones, partner-facing APIs, internal analytics services, and settlement flows with stronger retention and audit requirements. The domain suits the problem because it combines strict compliance constraints with heterogeneous implementation realities:

- PCI-sensitive communication paths require stronger cryptographic guarantees.
- some internal analytics services already support hybrid or pure post-quantum modes.
- external and legacy dependencies still pin parts of the workflow to transitional configurations.

Deployment. Operators defined policy resources for service tiers rather than configuring protocol details directly: cardholder-data services require quantum-safe

communication where feasible, while internal services prefer hybrid operation to preserve compatibility and rollout flexibility. We then injected three disturbances: delayed capability updates on selected services, a simulated rollout rejection for one settlement dependency, and a policy tightening for cross-zone traffic.

Outcomes. QuantumTrustKG derived a mixed deployment: pure post-quantum profiles on the most sensitive paths, hybrid configurations on the majority of internal interactions, and explicit flags on legacy edges that need migration planning. All PCI-sensitive paths received compliant configurations. The [P] setup-cost estimate stayed below 10% relative to the classical baseline (9.4% on the comparable Online Boutique run), and the injected settlement rollback was contained to the affected dependency chain without forcing a global policy downgrade.

Operators specified intent at the tier level, and the controller derived the edge-specific profiles and refusals without requiring per-connection mesh rules.

10 Discussion, Limitations, and Threats to Validity

Guarantee boundary. Theorems 1–2 constrain the *declared control-plane meta-data* rather than the bytes negotiated on the wire. A compromised workload could advertise quantum-safe capability and then negotiate classical TLS, and the proofs would still hold while the connection was in fact unprotected. The same boundary applies to the *declared-label* policy compliance metric (Sec. 8.2). The integrity of the annotation layer rests on Kubernetes RBAC separating capability publishers, policy authors, and the status writer. An authorized principal can still forge metadata. Binding those declarations to *observed* wire negotiation is the next validation step (§12).

Cost-weight guidance. Because admissibility is decided before ranking, the weights (w_o, w_r, w_m, w_h) and the hysteresis δ cannot affect safety (Prop. 2). This makes QTPO a two-level instance of priority-first selection [42]. The weights express an operational preference among already-safe options. Retuning changes profile choice without changing the admissibility test. The terms have the following operational interpretations.

Start from the risk term. w_r is the only weight that tracks a security quantity (the class gap, the boundary sensitivity, and advisory severity). It should dominate—our default $w_r=0.40$ is the largest weight. Raising it biases the tie-break toward the strongest admissible profile wherever several qualify, at the cost of more churn. Lowering it favors lower-cost admissible profiles without changing which profiles pass the gates.

Use w_m and w_h to control rollout pace rather than posture. MigrationCost and SwitchPenalty govern how eagerly the controller moves an edge that is already acceptable. An operator mid-migration, with capability claims still churning, should raise w_h and δ . One running a scheduled cut-over window should lower both so that promotions land promptly once endpoints become capable. δ is an absolute gap on the weighted score: our $\delta=0.05$ suppresses switches whose score improvement is at most 0.05. On the H2 meshes no score gap between an active

profile and a better competitor falls within $(0, 0.075]$. Varying δ between 0 and 0.075 changes no selection there (Appendix E.2), its effect depends on how close a deployment’s candidate scores are.

Treat w_o as the budget dial. Overhead is the only term fed by the setup-cost model, and that model is the least grounded input in the paper (Sec. 8.6). Operators with measured costs for their own environment should substitute them. Operators without should keep w_o small, as we do (0.25), ensuring that a modelled quantity has limited influence over the outcome.

Validate with a sweep. On the H2 meshes a $\pm 50\%$ sweep of each weight and of δ changes the promoted profile on up to 20.5% of promoted edges but leaves $C_{\text{decl}}^{\text{harness}}$ at 96.5% and unsafe promotions at zero [M] (Sec. 8.8). The sweep is cheap to re-run on a deployment’s own fixture and is the direct way to check that a weight change has moved only the tie-break. Under Prop. 2, a nonzero unsafe-promotion rate in such a sweep would indicate that the implementation no longer matches the filter-before-rank rule.

Multi-cluster implications. We measure one cluster and separate those results from the following multi-cluster architectural analysis.

Edge-local behavior. The decision unit is a call edge, and the gates are edge-local functions of that edge’s facts. Nothing in Def. 1 or Algorithm 2 refers to a cluster. A per-cluster controller deciding the edges whose destination it owns would compute the same assignment for those edges that a single controller does. The complexity result (Prop. 2(ii)) partitions with the edge set.

Federation requirements. (i) *Cross-cluster edges need an owner.* An edge whose source and destination sit in different clusters is decided by whichever controller sees it. If both do, they must agree, and the natural rule—destination-owned, since the destination’s `PeerAuthentication` enforces the outcome—has to be stated rather than assumed. (ii) *Freshness windows become network-dependent.* Our 300s window bounds staleness within one cluster’s watch pipeline. Across clusters, a capability claim must be replicated before it can be read. The effective window is the declared one minus replication delay. We would expect an operator to shorten the window. By Thm. 2, additional delay then produces more blocked-safe edges while preserving the unsafe-promotion result. This behavior follows from the theorem but has not been measured in a federated deployment. (iii) *Partition behaviour is the open question.* During a partition, each side sees facts it cannot refresh. The fail-closed rule already covers this case—an unrefreshable fact blocks—so the expected failure mode is a burst of blocked edges on both sides followed by convergence delay after recovery. The quantity we cannot predict is *how large* that burst is, because it depends on the cross-cluster edge fraction and on replication latency, neither of which we measured.

A federated testbed with injected partitions is the next experiment. It should vary the cross-cluster edge fraction and partition duration.

Scope and operational boundaries. (1) Topology is annotation-discovered rather than telemetry-driven. Telemetry should enter as provenance-qualified evidence rather than as authority. (2) Post-quantum setup costs are modelled rather than measured, apart from the approximately 1% handshake term. Sec. 8.6

Table 15. Capabilities explicitly implemented in the cited configurations: yes (✓), partial (∼), no (✗), or unreported (?). ABAC = attribute-based access control.

Approach	Edge gran.	Prov. gated	Hard gate	Fail closed	Mesh config.	Native PQ wire
Static mesh / Istio config	✗	✗	✗	∼	✓	✗
Policy-as-code (OPA) [11,12]	?	?	✓	∼	∼	✗
Service-mesh ABAC [7]	∼	?	✓	∼	✓	✗
Zero-trust / partial-trust mesh [8,9]	✗	∼	✓	✓	✓	✗
KG network compliance [14]	✗	∼	∼	✗	✗	✗
Self-adaptive microservices [21]	∼	✗	∼	∼	∼	✗
Service-level crypto-agility (SL-CA)	✗	✗	∼	✗	✓	✓
Enterprise agility (ELCA) [34]	✗	✗	✓	✗	✓	✓
Config. profiling [35]	✗	✓	∼	✓	∼	✓
Opportunistic negotiation (OPP)	✓	✗	✗	✗	✓	✓
QuantumTrustKG (ours)	✓	✓	✓	✓	✓	✗

Columns: edge-granular decisions, provenance gating, hard policy gates, fail-closed behavior, mesh configuration, and native PQ wire negotiation.

gives the full derivation and Table 8 marks every input. (3) All four competing systems are our re-creations of published decision rules. Only the QTPO-NoKG ablation is a fully controlled comparison, and it shows the smallest gap (Sec. 8.4). (4) The real-topology subgraphs are hub-centred breadth-first neighbourhoods and are more concentrated than the full trace. The 37–60% band describes the hub-dominated part of a production system (Sec. 8.5). (5) The semantic path uses Go checks over SPARQL results rather than an integrated Web Ontology Language (OWL)/SHACL runtime. We evaluate edge-local selection rather than global optimization. (6) If the graph store or the watch pipeline is unavailable, the controller fails closed. Prolonged staleness degrades availability but never safety.

Threats to validity. *Internal:* the Coq predicate and the Go checker mirror each other field-for-field and are differentially tested, but that correspondence is not itself machine-checked, the in-cluster (H1) numbers were recorded with an earlier controller release and are replayed rather than re-measured (Sec. 8.1), an insider editing a policy CRD remains the residual risk RBAC does not close. *External:* workloads are controlled testbeds plus one Alibaba-topology subset, and the 500–2000-service rows are semantic-core stress *excluding* Fuseki, watches, and Istio churn. The grounded end-to-end claim stays the 50–200-service measurement. Everything is measured in one cluster. *Construct:* the headline metric is computed from declared labels and therefore cannot detect a workload that lies about its capability, adversarial annotations and mass staleness convert to blocked-safe with zero unsafe promotions in the evaluated workloads.

11 Related Work

Table 15 reports features explicitly implemented in the cited configurations. A cross does not claim that the underlying language or platform cannot express a

feature. Question marks indicate that the cited configuration did not specify or evaluate it.

Service composition and orchestration. Classical service composition and quality-of-service (QoS)-aware selection [18,16,17,19] rank candidate services by cost or quality under a *fixed* correctness posture. Microservice orchestration and adaptive control [20,21] choose placement, scale, or traffic shape from quality and vulnerability signals. We reuse the control-loop pattern, but the decision variable is whether a cryptographic profile is *admissible* on an edge, where refusing the edge is a legitimate answer. Unlike QoS ranking, admissibility is the primary gate and cost breaks ties only among candidates that have already passed it. The safety property is therefore independent of the ranking weights (Prop. 2), which a pure ranking formulation cannot offer.

Policy and governance. Policy-as-code (OPA/Gatekeeper) [11,12], service-mesh attribute-based access control (ABAC) [7], externalized authorization middleware [6,22], and partial- or zero-trust control planes [8,9,10] the cited configurations enforce a posture supplied as input and are not PQC-aware. They do not report the fresh edge-local state used here. Runtime guards for LLM-driven service composition [43] share the fail-closed stance and block any action that their policy automaton refuses. QTPO additionally records a structured promotion-block reason while retaining any previous active profile. Admission-control webhooks and intent-based networking share the gap: they validate writes or render intent, without choosing a per-connection profile under fresh provenance.

Crypto-agility-specific systems are the nearest comparison. Enterprise-level crypto-agility (ELCA [34]) drives cryptography from *per-service* policy. Automated configuration profiling [35] builds a *per-endpoint* inventory with provenance but cannot express per-edge divergence, and Cryptography Bills of Materials (CBOM [41]) standardize a *static* inventory. Per service, per endpoint, or static: none decides a per-edge posture under fresh provenance, which is the layer QuantumTrustKG adds. We measure ELCA, configuration profiling, SL-CA, and OPP as baselines in Sec. 8, and state in Sec. 8.4 how those re-creations were built.

Semantic and graph methods [13,15,23,14,24,25,27,28] supply the modelling vocabulary for discovery, conformance, and deontic conflict. We keep the ontology small and operational, and we quantify the cost of the RDF choice against property-graph and relational encodings with a same-join microbenchmark (Sec. 4, Table A4) rather than asserting it.

Cryptographic layer. The NIST post-quantum standards [29], hybrid TLS guidance [30,31], and post-quantum TLS cost studies [32,33] define and measure the profile classes themselves. QuantumTrustKG consumes those definitions and decides, per edge and under provenance, which of them to deploy.

12 Conclusion

Post-quantum migration is not a single switch for an entire service. Each communication edge combines a peer capability, a policy obligation, fresh evidence, and

rollout state. We built QuantumTrustKG around that observation. Its graph-backed control plane joins these facts, QTPO filters inadmissible profiles before comparing costs, and the Kubernetes/Istio controller records the accepted assignment as auditable mesh configuration. Because ranking begins only after admission, cost preferences cannot override the control-plane gates.

The controlled comparison records zero unsafe promotions and 96.5% declared-label policy compliance. Re-created service-level and opportunistic baselines record 3.4–13.7% unsafe promotions on synthetic meshes, and service-level and per-endpoint baselines record 37–60% on sampled production topologies, while the semantic-core experiment extends the decision path to 2000 services. Together, these results show that edge granularity and evidence-aware admission matter most when endpoint capabilities and policy obligations diverge.

The remaining step is to connect a justified control-plane decision to the cryptography actually negotiated on the wire. Our compliance metric evaluates the declarations consumed by the controller, the in-cluster replay comes from one cluster and an earlier release, and most aggregate setup cost is modelled apart from the measured approximately 1% handshake term. Wire-level validation and attested capability binding to workload identity and Secret Discovery Service (SDS) material would complete that evidence chain.

Acknowledgments. We acknowledge Ho Chi Minh City University of Technology (HCMUT), VNU-HCM for supporting this study.

References

1. L. Gasser, “Post-Quantum Cryptography,” in *Trends in Data Protection and Encryption Technologies*. Cham: Springer, 2023, pp. 47–52.
2. D. Ott, C. Peikert, et al., “Identifying Research Challenges in Post Quantum Cryptography Migration and Cryptographic Agility,” *arXiv:1909.07353*, 2019.
3. N. Bindel, M. Mosca, and B. Munson, “The Quantum Threat to Cybersecurity and Privacy,” in *The Boundaries of Data*. Amsterdam: Amsterdam University Press, 2024, pp. 35–52.
4. Y. Cao et al., “The Evolution of Quantum Key Distribution Networks: On the Road to the Qinternet,” *IEEE Commun. Surv. Tutor.*, vol. 24, no. 2, pp. 839–894, 2022.
5. A. Venckauskas et al., “Enhancing Microservices Security with Token-Based Access Control Method,” *Sensors*, vol. 23, no. 6, Art. no. 3363, 2023.
6. M. Sauwens et al., “ThunQ: A Distributed and Deep Authorization Middleware for Early and Lazy Policy Enforcement in Microservice Applications,” in *Proc. ICSOC 2021*, 2021, pp. 204–220.
7. R. Chandramouli, Z. Butcher, and A. Chetal, “Attribute-Based Access Control for Microservices-Based Applications Using a Service Mesh,” NIST Special Publication 800-204B, Aug. 2021.
8. C. Adam et al., “Partially Trusting the Service Mesh Control Plane,” *arXiv:2210.12610*, 2022.

9. A. Poudel et al., “Mazu: A Zero Trust Architecture for Service Mesh Control Planes,” in *Proc. EuroSec*, 2025, pp. 49–55.
10. A. Asad, P. McClurg, and J. Andrade, “C8s: A Confidential Kubernetes Architecture,” *arXiv:2604.26974*, 2026.
11. A. P. Jothimani, “Enabling Secure Cloud Governance Using Policy as Code,” M.S. thesis, Chalmers Univ. of Technology and Univ. of Gothenburg, Sweden, 2022.
12. S. Pallewatta and M. A. Babar, “Towards Secure Management of Edge-Cloud IoT Microservices Using Policy as Code,” in *Proc. ECSA 2024*, 2024, pp. 270–287.
13. S.-P. Ma, H.-J. Lin, and M.-J. Hsu, “Semantic RESTful Service Composition Using Task Specification,” *Int. J. Softw. Eng. Knowl. Eng.*, vol. 30, no. 6, pp. 835–857, 2020.
14. A. Simic and D. Palma, “Using Knowledge Graphs to Automate Network Compliance of Containerized Services,” in *Proc. CNSM*, 2024, pp. 1–5.
15. T. V. Vignesh, K. Vijaya, and R. I. Minu, “Graph-Theoretic Approaches in Semantic Web Service Orchestration and Discovery: A Comprehensive Literature Survey,” in *Proc. ICDICI*, 2024, pp. 1026–1029.
16. A. Marchese and O. Tomarchio, “Network SLO-Aware Container Orchestration on Kubernetes Clusters,” in *Proc. ICSOC 2024*, 2025, pp. 96–104.
17. H. Park et al., “Optimizing Traffic Allocation for Multi-Replica Microservice Deployments in Edge Cloud,” in *Proc. ICSOC 2024*, 2025, pp. 100–115.
18. N. Li et al., “SCORE: A Resource-Efficient Microservice Orchestration Model Based on Spectral Clustering in Edge Computing,” in *Proc. ICSOC 2022*, 2022, pp. 186–202.
19. D. Kim et al., “LARE-HPA: Co-Optimizing Latency and Resource Efficiency for Horizontal Pod Autoscaling in Kubernetes,” in *Proc. ICSOC 2024*, 2025, pp. 19–34.
20. V. Riccio et al., “Engineering Self-Adaptive Microservice Applications: An Experience Report,” in *Proc. ICSOC 2023*, 2023, pp. 227–242.
21. M. Camilli et al., “Integrated QoS- and Vulnerability-Driven Self-Adaptation for Microservices Applications,” in *Proc. ICSOC 2024*, 2025, pp. 55–71.
22. N. Saenger and S. Abeck, “Externalized and Decentralized Authorization of Microservices,” in *Proc. ICSOC 2024*, 2025, pp. 182–193.
23. K. Shi et al., “BSDG: Anomaly Detection of Microservice Trace Based on Dual Graph Convolutional Neural Network,” in *Proc. ICSOC 2022*, 2022, pp. 171–185.
24. S. Giallorenzo et al., “A Toolchain for Checking Domain- and Model-Driven Properties of Jolie Microservices,” in *Proc. ICSOC 2024*, 2025, pp. 161–175.
25. D. F. Holtebo et al., “EXOGEM: Extending OpenAPI Generator for Monitoring of RESTful APIs,” in *Proc. ICSOC 2022 Workshops*, 2023, pp. 118–129.
26. GoogleCloudPlatform, “Online Boutique: Cloud-First Microservices Demo Application,” GitHub repository, accessed May 11, 2026.

27. A. Massari and S. Peroni, “Performing Live Time-Traversal Queries via SPARQL on RDF Datasets,” *arXiv:2210.02534v2*, 2022.
28. L. Robaldo and G. Pozzato, “Handling Irresolvable Conflicts in the Semantic Web: An RDF-Based Conflict-Tolerant Version of the Deontic Traditional Scheme,” *J. Log. Comput.*, vol. 35, no. 8, Art. no. exaf054, 2025.
29. National Institute of Standards and Technology, “Module-Lattice-Based Key-Encapsulation Mechanism Standard” (FIPS 203), “Module-Lattice-Based Digital Signature Standard” (FIPS 204), and “Stateless Hash-Based Digital Signature Standard” (FIPS 205), Aug. 2024.
30. K. Kwiatkowski, P. Kampanakis, B. E. Westerbaan, and D. Stebila, “Post-Quantum Traditional (PQ/T) Hybrid Key Agreement Mechanisms for TLS 1.3,” RFC 10024, Aug. 2026.
31. ETSI, “CYBER; Quantum-Safe Cryptography (QSC); Quantum-Safe Hybrid Key Establishment,” TS 103 744 V1.2.2, Feb. 2026.
32. P. Schwabe, D. Stebila, and T. Wiggers, “Post-Quantum TLS Without Handshake Signatures,” in *Proc. ACM CCS*, 2020, pp. 1461–1480.
33. D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, “Post-Quantum Authentication in TLS 1.3: A Performance Study,” in *Proc. NDSS*, 2020.
34. D. Sikeridis et al., “ELCA: Introducing Enterprise-level Cryptographic Agility for a Post-Quantum Era,” *IACR Cryptology ePrint Archive*, Report 2023/1539, 2023.
35. H. Balaji et al., “Operationalising Post Quantum TLS Automated Configuration Profiling and Hybrid PQC Deployment in Financial Infrastructure,” *arXiv:2605.17955*, 2026.
36. Alibaba Group, “Alibaba Cluster Trace Program: cluster-trace-microservices-v2022,” 2022. <https://github.com/alibaba/clusterdata>; see S. Luo et al., “Characterizing Microservice Dependency and Performance: Alibaba Trace Analysis,” *Proc. ACM SoCC*, 2021.
37. Y. Gan et al., “An Open-Source Benchmark Suite for Microservices and Their Hardware-Software Implications for Cloud & Edge Systems,” in *Proc. ASPLOS*, 2019, pp. 3–18.
38. X. Zhou et al., “Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study,” *IEEE Trans. Software Engineering*, vol. 47, no. 2, pp. 243–260, 2021.
39. J. Hesse and M. Rosenberg, “PAKE Combiners and Efficient Post-Quantum Instantiations,” in *Advances in Cryptology—EUROCRYPT 2025*, LNCS 15602. Cham: Springer, 2025, pp. 395–420.
40. W. Beullens, L. Dodgson, S. Faller, and J. Hesse, “The 2Hash OPRF Framework and Efficient Post-Quantum Instantiations,” in *Advances in Cryptology—EUROCRYPT 2025*, LNCS 15608. Cham: Springer, 2025, pp. 332–362.
41. OWASP CycloneDX, “Cryptography Bill of Materials (CBOM),” CycloneDX Specification v1.6, 2024. <https://cyclonedx.org/capabilities/cbom/>.
42. X.-B. Le et al., “Lexi: Priority-First Service Orchestration,” in *Proc. ICSOC*, LNCS, Springer, 2026, to appear.

43. X.-B. Le et al., “Deon: Runtime Guards for Policy-Safe Agentic Service Composition under Attack,” in *Proc. ICSOC*, LNCS, Springer, 2026, to appear.
44. M. Mosca, “Cybersecurity in an Era with Quantum Computers: Will We Be Ready?,” *IEEE Security & Privacy*, vol. 16, no. 5, pp. 38–41, 2018.
45. C. Paquin, D. Stebila, and G. Tamvada, “Benchmarking Post-Quantum Cryptography in TLS,” in *Proc. PQCrypto*, LNCS, Springer, 2020, pp. 72–91.
46. A. Hogan et al., “Knowledge Graphs,” *ACM Computing Surveys*, vol. 54, no. 4, pp. 1–37, 2021.

A Artifact Map: Which Command Produces Which Claim

The appendices that follow are part of this document rather than a separate supplement. Every claim in the main text can be checked against the material here. This first section maps each quantitative claim to the artifact command that produces it and the output file that records it.

Table A1. Claim-to-artifact map. Commands are run from the artifact root. Outputs land under `implementation/experiments/outputs/processed/`.

Claim (main text)	Command	Output file
Table 4 (H2 comparison)	<code>make -C implementation baseline-compare</code>	<code>baseline-comparison.csv</code>
Table 6 (real topologies)	<code>same target, real-topology scenario over fixtures/real-topologies/alibaba</code>	<code>baseline-comparison-real-alibaba.csv</code>
Real-topology structure (fan-in shares)	<code>read directly from the *.edges fixture files</code>	<code>fixtures/real-topologies/alibaba/</code>
Table 7 (mass staleness)	<code>make -C implementation staleness-sweep</code>	<code>staleness-sweep-real-alibaba.csv, staleness-sweep-results.csv</code>
Table 3 (in-cluster compliance)	<code>run-paper-results.py</code> (canonical replay: re-emits recorded values, no re-measurement)	<code>compliance-wilson.csv</code>
Table 11 (latency)	<code>run-paper-results.py</code> (replay)	<code>latency-results.csv</code>
Table 9, 8 (setup cost)	<code>run-paper-results.py</code>	<code>setup-cost-results.csv, setup-cost-perturbation.csv, oqs-calibration-bias.csv</code>
Table 10 (native TLS)	<code>run-pq-tls-handshake.py, run-wire-tls-validation.py</code>	<code>pq-tls-handshake-results.csv, wire-tls-validation.csv</code>
Table 12 (semantic core)	<code>go run ./cmd/scale-stress</code>	<code>scale-stress-results.csv</code>
Table 13, 14	<code>run-paper-results.py; unit tests under controller/tests/unit</code>	<code>fault-injection-results.csv, blocked-safe-breakdown.csv</code>
Weight sweep (§E.2)	<code>make -C implementation weight-sweep</code> (H2, regenerated)	<code>weight-sweep-results.csv</code>
Online Boutique (§C.2)	<code>go run ./cmd/decision-trace; baseline-compare</code> (online-boutique scenario)	<code>online-boutique-decision-trace.json, online-boutique-ablation.csv, online-boutique-rq-stats.csv</code>
Table A4 (data model)	<code>make datamodel-benchmark</code>	<code>datamodel-benchmark.csv</code>
Theorems 1–2, Prop. 1	<code>make -C formal/coq</code>	<code>formal/coq</code> (checked theorem names in §6)
Whole-artifact status report	<code>scripts/run-full-evidence.sh</code>	<code>evidence-summary.json, evidence-summary.md</code>

The evidence summary reports each check as passed, skipped, unavailable, or failed, and never converts a missing dependency into an inferred measurement. Two optional dependencies were absent on our host and are reported as such: `kind`, which gates the full-controller scale and churn rows (Table A2), and an `oqs-provider` installation, which is not required because stock OpenSSL 3.6.2 already exposes ML-KEM natively. The full timing columns of Table A2 are populated only under a `kind` environment with the controller, Fuseki, and the CRDs installed. We record those rows as unavailable rather than extrapolating them.

Table A2. Controller scale/churn harness availability (timing requires `kind`).

Harness	Services/rate	Unsafe pro- motions	Local status
Scale	50/100/200/500 services	n/a (not run)	Not run locally because <code>kind</code> is unavailable; processed rows preserve expected edge, Istio-object, and status-writeback counts
Churn	50/100/250 updates per minute at 200 and 500 services	n/a (not run)	Not run locally because <code>kind</code> is unavailable; timing, backlog, and convergence columns require a full <code>kind</code> run

B System and Trust Details

The motivating deployment is a payment-processing application with frontend, authentication, payment, ledger, and audit services. These services do not migrate at the same pace. The frontend may still depend on partner-facing or legacy clients and therefore support only classical or hybrid profiles, while the ledger and audit services may require quantum-safe communication because they protect long-lived financial records. The correct protocol assignment is therefore edge-specific: frontend-to-authentication may remain transitional, authentication-to-payment may use a hybrid profile while endpoint capability is uneven, and payment-to-ledger should use a quantum-safe profile when feasible because it carries regulated and long-lived data.

The application model abstracts this setting as a directed service graph $G = (\mathcal{S}, E)$. Each service records declared cryptographic capabilities, sensitivity labels, compliance obligations, and migration state. Each edge records its trust boundary, active policy, current assignment, rollout status, and the provenance of every fact the controller uses. Protocol profiles record key-establishment primitive, authentication primitive, primitive family, security assumption, proof status, declared security class, and deployment cost. Proof status is metadata for policy and audit rather than a primitive-level proof.

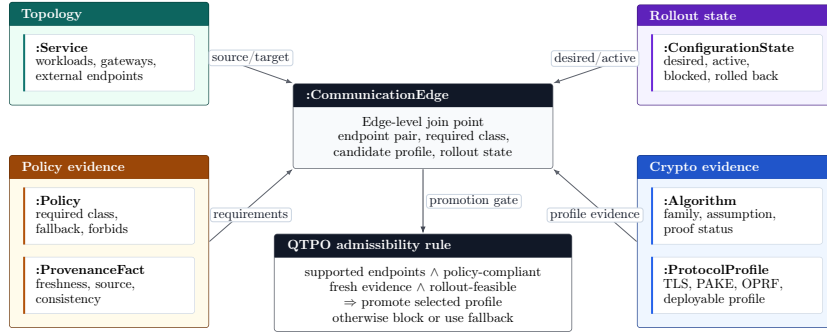


Fig. A1. Edge-centered ontology fragment used by QuantumTrustKG to join topology, policy and provenance, cryptographic evidence, and rollout state before QTPO promotion.

Trust boundaries. The controller trusts the Kubernetes API server, the Istio control plane, and its CRD store as the operational control plane. Operator-supplied call annotations, workload-reported capability claims, and threat advisories are treated as untrusted observations unless they carry a source, an observation time, and freshness metadata. A mis-annotation, a stale capability claim, a contradictory advisory, or a graph outage cannot create a new promotion. It can only block one or preserve the last committed assignment. A compromised operator holding valid Kubernetes credentials can weaken policy intent, but cannot bypass the admissibility filters applied to the resulting graph state.

We separate what the design covers from what it does not, beyond the RBAC boundary.

False capability advertisement. A workload over-claiming `quantum_safe` support cannot cause a downgrade—promotion never lowers the required class—but it could induce a promotion the endpoint then fails to honor. This is why capability is an observation gated on freshness and provenance, and why feeding the negotiated group back as provenance (the “wire loop”) is the natural hardening rather than a claim of the current artifact. It is also precisely the gap that names the headline metric *declared-label* policy compliance (§8.2).

Annotation and label tampering. Tampering with `quantumtrustkg.io/calls` or `quantumtrustkg.io/trust-boundary` could add or relax an edge. The mass-staleness sweep (Table 7) quantifies the fail-closed response: injected corruption converts to blocked-safe with zero unsafe promotions. Tampering degrades availability rather than safety.

Policy-provenance manipulation inside a trusted component, such as an insider editing a policy CRD to relax a regulated edge, is the residual risk RBAC alone does not close. An authorized policy author can still weaken intent. We therefore treat separation of the capability-publisher, policy-author, and controller-owned status-writer roles under RBAC as an assumption, and signed or attested policy

and capability provenance as the next hardening step rather than a guarantee of this artifact.

C Implementation Notes

The prototype discovers services from Kubernetes, reading `quantumtrustkg.io/calls` for communication edges and `quantumtrustkg.io/trust-boundary` for trust zones. Labels classify internal, partner-facing, and regulated flows. This avoids a dependence on any particular telemetry backend while still exercising the orchestration path. Envoy access logs, OpenTelemetry traces, or Service Mesh Interface (SMI)-style traffic descriptions could be added as observations, but would remain provenance-qualified evidence rather than authoritative topology.

When Fuseki is reachable, the controller synchronizes the cluster snapshot through SPARQL updates and queries active policies and candidate profiles. During a graph outage it pins the last committed status object, blocks new promotions that cannot be justified from fresh local state, and lets the Kubernetes work queue retain reconciliation keys. After recovery it resynchronizes from Kubernetes Services, CRDs, and status objects before replaying pending reconciliations.

For each edge, QTPO computes admissible profiles from endpoint support, policy-derived minimum levels, trust-boundary propagation, and deprecated-profile restrictions. Promoted edges are materialized as `PeerAuthentication` and `DestinationRule` objects, then verified in manager mode. Outcomes are written to `AssignmentStatus` resources recording desired profile, active profile, assignment state, and failure reason. The controller supports immediate, staged, and canary-style rollout windows. A batch advances only after the generated Istio resources are present and both source and destination workloads are ready.

C.1 Profile-to-Mesh Manifest Walkthrough

Each promoted edge is materialized as exactly two Istio objects carrying the selected profile as control-plane metadata. For the running `Checkout→Payment` example (required class `quantum_safe`, selected `QuantumSafeProfile`) the controller emits the pair below. The `PeerAuthentication` pins strict mTLS on the destination workload. The `DestinationRule` sets the mesh TLS mode and carries the annotations that make the decision auditable. Stock Envoy/Istio enforces mTLS and records this metadata, but does not itself negotiate the declared post-quantum group on the wire—the control-plane/data-plane boundary restated throughout this paper.

```
kind: PeerAuthentication
metadata:
  name: checkoutToPayment-mtls
  labels:      {managed-by: qtkg}
  annotations: {edge-id: checkoutToPayment,
                profile: QuantumSafeProfile,
```

```

        security-level: quantum_safe}
spec:
  selector: {matchLabels: {app: Payment}}
  mtls: {mode: STRICT}
---
kind: DestinationRule
metadata:
  name: checkoutToPayment-tls
  labels:      {managed-by: qtkg}
  annotations: {edge-id: checkoutToPayment,
                  profile: QuantumSafeProfile,
                  trust-boundary: regulated}
spec:
  host: Payment.<ns>.svc.cluster.local
  trafficPolicy: {tls: {mode: ISTIO_MUTUAL}}
```

(apiVersions are `security.istio.io/v1beta1` and `networking.istio.io/v1beta1`, annotation keys carry the `quantumtrustkg.io/` prefix, managed-by the `app.kubernetes.io/` prefix.)

C.2 Online Boutique Fixture and Decision Trace

The artifact ships a deterministic Online Boutique fixture encoding the standard frontend, checkout, payment, cart, Redis cart, product catalog, currency, shipping, email, recommendation, and ad services with their declared trust boundaries. The main trace is **Checkout→Payment**: PCI policy and the regulated boundary derive `quantum_safe`, the hybrid profile is rejected before ranking, the quantum-safe profile is selected, and the emitted object names are `checkoutToPayment-mtls` and `checkoutToPayment-tls` (Table A3). The finance fixture of §9 remains a domain case study rather than the realism anchor.

Table A3. Online Boutique decision trace shipped with the artifact.

Trace field	Recorded value
Edge	<code>checkoutToPayment: Checkout -> Payment</code>
Requirement	<code>quantum_safe</code> from PCIPolicy, PCI context, and regulated boundary
Candidate removed	<code>HybridProfile</code> , rejected by the policy/security-class gate
Selected candidate	<code>QuantumSafeProfile</code> , score 0.4675 under the default weights, after validity filters
Generated names	Istio <code>checkoutToPayment-mtls; checkoutToPayment-tls</code>
Boundary	The objects enforce strict mTLS and carry profile metadata in stock Istio; they do not negotiate post-quantum TLS without a post-quantum-capable data plane

C.3 Data-Model Microbenchmark

Table A4 answers the identical per-edge admissibility join over the 1000-service fixture (2000 edges) under three encodings, reporting the median of 5 repeats, load time includes Turtle parsing. All three return the same 1889 admissible

edges. The relational (SQLite) and property-graph (networkx) encodings answer in about 10 and 7 ms, while rdflib/SPARQL takes roughly 1 s—the query-latency price of native per-statement provenance and an open schema. Because selection runs off the data path, the control loop absorbs that cost. The number quantifies the trade-off argued in Sec. 4, and the deployed store is Fuseki rather than rdflib.

Table A4. Data-model microbenchmark: the same admissibility join, 1000-service fixture, median of 5 repeats.

Engine	Load (ms)	Query (ms)	Total (ms)	Provenance
Property graph (networkx)	5.9	0.95	6.8	partial (node/edge properties)
Relational (SQLite)	7.1	2.5	9.7	bolt-on (extra tables)
RDF/SPARQL (rdflib)	95.3	918.4	1013.8	native (named graphs)

D Ontology, Queries, and Profile Sources

This section records the operational detail of the semantic control-plane model of Sec. 4: the ontology constructs the controller relies on, the questions the model has to answer, the queries that answer them, and the external sources behind each protocol-profile class.

Table A5. Core ontology constructs from the Turtle ontology and their orchestration roles.

Construct	Represents	Key relations	Role in orchestration
:Service	Workloads, gateways, endpoints	gate-:supports, external:dependsOn, :subjectTo	:supportsProfile, Supplies endpoint capability and policy scope
:Communication	Directed service-to-service interaction	:sourceService, :destinationService, :crossesBoundary, :assignedProtocol	Makes protocol assignment edge-specific rather than cluster-wide
:Algorithm	Classical, hybrid, and post-quantum primitives	Reached from :supports	Records primitive support when the artifact models algorithm-level claims
:Protocol	Deployable and related profile	TLS, :hasSecurityCategory, PAKE, OPRF, or :hasOverheadScore	Candidate unit selected by QTPO
:Policy	Compliance, trust, and data-handling intent	:requiresLevel, :subjectTo links	Derives edge-level minimum security and restrictions
:Observation	Source, freshness, and metadata	:observedBy, :observedAt, consistency :freshUntil, :conflictNote	Blocks stale or contradictory assignments
:Configuration	Desired and active assignment state	:hasDesiredAssignment, :hasActiveAssignment	Connects semantic selection to migration drift and rollout status

Table A6. Competency questions the semantic control-plane model must answer.

Question	Required facts	Expected controller use
Candidate profile discovery	<code>:Communication</code> , endpoint services, <code>:Protocol</code> , security category, overhead score	Build the candidate set before policy and threat filtering
Strongest requirement derivation	Service <code>:subjectTo</code> policies, <code>:requiresLevel</code> , trust boundary	Derive <code>Req(e)</code> from policy, boundary, and default rules
Stale-fact blocking	<code>:Observation</code> , <code>:observedAt</code> , <code>:freshUntil</code> , <code>:conflictNote</code>	Reject expired or contradictory evidence before ranking
Migration-drift detection	<code>:Configuration</code> , <code>:hasDesiredAssignment</code> , <code>:hasActiveAssignment</code>	Identify edges where desired and active profiles differ

Table A7. Representative SPARQL patterns shipped with the artifact.

Query	Representative pattern	Purpose
candidate protocols	<code>?comm a :Communication;</code> <code>:sourceService ?src;</code> <code>:destinationService ?dst.</code> <code>?protocol a :Protocol;</code> <code>:hasSecurityCategory ?category;</code> <code>:hasOverheadScore ?overhead.</code>	Discover deployable profile candidates and ranking inputs
active policies	<code>?service a :Service; :subjectTo</code> <code>?policy. ?policy :requiresLevel</code> <code>?level.</code>	Retrieve policies that feed requirement derivation
stale facts	<code>?observation a :Observation;</code> <code>:freshUntil ?freshUntil.</code>	Surface observations that must be compared with controller time
migration edges	<code>?config a :Configuration;</code> <code>:hasDesiredAssignment ?desired;</code> <code>:hasActiveAssignment ?active.</code> <code>FILTER(?desired != ?active)</code>	Find desired/active drift during rollout

Table A8 lists the external sources behind each protocol-profile class. The framework consumes these declarations and does not re-prove any of them (§6).

The SHACL files in the artifact’s constraints directory are validation aids for shape-checking ontology data and fixtures. The runtime controller enforces the corresponding gates through explicit Go checks and does not claim a full SHACL execution engine. Sec. 10 lists integrating one as future work.

Table A8. Protocol-profile theory sources used by QuantumTrustKG.

Profile family	Source	Assumption/status	Role in QuantumTrustKG
ML-KEM	NIST FIPS 203 [29]	Standardized	Post-quantum key establishment
ML-DSA / DSA	SLH- NIST FIPS 204/205 [29]	Standardized	Post-quantum authentication metadata
Hybrid TLS	RFC 10024 [30]	Standards-track	Transitional mesh profile
Hybrid key establishment	ETSI TS 103 744 [31]	Standard guidance	Combiner metadata
Hybrid PAKE	Eurocrypt 2025 [39]	Top-venue proven	Optional authenticated profile class
Quantum-safe OPRF	Eurocrypt 2025 [40]	Top-venue proven	Optional privacy-preserving profile class
Post-quantum natures	sig- NIST FIPS 204/205 [29]	Standardized	Assumption and proof-status context

E Supplementary Evaluation Detail

The evidence that the main claims rest on is in Sec. 8. This section adds the supporting detail: how the scoring terms behave, the weight and hysteresis sweeps in full, the specific cases that separate QTPO from a greedy selector, what the graph-backed model contributes capability by capability, the controller’s behaviour across failure modes, and the observed resource footprint.

E.1 QTPO Scoring Terms

Table A9 illustrates each scoring term. Validity filters dominate ranking: the score orders only candidates that already satisfy the support, policy/threat, provenance, rollout, and mesh-readiness gates.

Table A9. QTPO scoring terms and their effect on ranking.

Term	Concrete example	Effect	Boundary
Risk(p, e)	A regulated payment edge gives any sub-quantum_safe candidate a prohibitive penalty	Keeps higher-risk edges from selecting weaker profiles	Cannot rescue an inadmissible candidate; the filter already removed it
Migration cost	Switching an active hybrid edge to pure post-quantum adds operational cost during staged rollout	Prefers lower-disruption options among valid candidates	Does not override security-class requirements
Switch penalty	If the active profile is within the hysteresis margin, keep it	Reduces profile churn under capability flapping	A true policy upgrade still forces recomputation
Hysteresis δ	A score improvement of at most 0.05 (absolute) is ignored; a larger one can trigger a switch	Stabilizes repeated reconciliations	Applied after admissibility and before mesh materialization

E.2 Weight Sensitivity and Hysteresis

We scale each QTPO weight (w_o, w_r, w_m, w_h) and the hysteresis δ by 0.5 and 1.5 around the operating point (0.25, 0.40, 0.20, 0.15; 0.05), one at a time, and also set $\delta=0$. The sweep runs the controller’s selection function on the 200-service H2 meshes (11,929 managed edges over 30 repetitions, seed 20260509). Half of the edges, chosen by seed, carry an existing active profile so that the migration-cost and switch terms and δ take effect. Table A10 gives the result. Declared-label policy compliance is 96.47% and unsafe promotions are zero at every point, as Prop. 2(i) requires: the filter chain produces compliance and does not read the weights. The weights do change *which* admissible profile is promoted—on 20.48% of promoted edges when w_o is halved or w_r raised by half, since both favour the quantum-safe profile over the cheaper hybrid, and on 0.04% for the opposite moves—while w_m, w_h and $\delta \in [0, 0.075]$ change no selection on these meshes: no score gap between an active profile and a better competitor falls within $(0, 0.075]$ there, the active profile is kept on 73.40% of the edges that have one at every point. We did not tune δ per testbed.

Table A10. Weight and hysteresis sweep on the 200-service H2 meshes (30 reps) [M].

Configuration	$C_{\text{decl}}^{\text{harness}}$ (%)	Unsafe (%)	Selection changed (%)
Operating point	96.47	0.00	—
$w_o \times 0.5$ / $w_o \times 1.5$	96.47 / 96.47	0.00 / 0.00	20.48 / 0.04
$w_r \times 0.5$ / $w_r \times 1.5$	96.47 / 96.47	0.00 / 0.00	0.04 / 20.48
$w_m \times 0.5$ / $w_m \times 1.5$	96.47 / 96.47	0.00 / 0.00	0.00 / 0.00
$w_h \times 0.5$ / $w_h \times 1.5$	96.47 / 96.47	0.00 / 0.00	0.00 / 0.00
$\delta \times 0.5$ / $\delta \times 1.5$ / $\delta=0$	96.47 (all)	0.00 (all)	0.00 (all)

E.3 Cases That Separate QTPO from a Greedy Selector

Table A11 lists representative cases behind the Greedy-vs-QTPO compliance gap at 200 services. QTPO-NoKG handles the two inheritance cases (cross-boundary inheritance and hybrid-fallback misuse) but not the two provenance cases (stale capability and contradictory advisory). Those are the gain attributable to the graph-backed model. Table A12 separates that contribution capability by capability: QTPO-NoKG keeps the same filter order, ranking, and hysteresis, but consumes flattened state rather than graph-backed provenance joins.

Table A11. Representative cases driving the Greedy-vs-QTPO compliance gap at 200 services.

Case	Description	Greedy outcome	QTPO outcome
Stale capability	Endpoint advertises quantum_safe but the claim is past its freshness window	Promotes quantum_safe	Blocks; reason stale-facts (caught by the graph; QTPO-NoKG also promotes)
Contradictory advisory	A family is simultaneously claimed supported and forbidden by separate sources	Promotes the cheaper option	Blocks; reason conflicting-facts (caught by the graph)
Cross-boundary inheritance	The edge crosses a regulated trust zone, raising $\text{Req}(e)$ above the greedy choice	Promotes a hybrid profile	Promotes quantum_safe (caught by both QTPO and QTPO-NoKG)
Hybrid-fallback misuse	Policy permits hybrid only when no post-quantum profile is feasible	Promotes hybrid for cost	Promotes post-quantum; the hybrid path is not entered (caught by both)

Table A12. The graph’s contribution, separated from the QTPO heuristic.

Capability	QTPO-NoKG view	Graph-backed view
Late joins	Pre-flattened labels and policy fields	Joins topology, policy, boundary, profile, rollout, and provenance at reconcile time
Named provenance	Source fields copied onto edge records	Source, observation time, and freshness remain attached to the justifying facts
Freshness windows	Only edge-level timestamps	Stale observations can be detected across fact sources before ranking
Contradiction detection	Local conflicts only if precomputed	Conflicting advisory and capability facts are visible to the controller’s query path
Decision-trace completeness	Explains filter and ranking state after flattening	Replays policy, facts, candidates, rejection reasons, final score, and materialized object names

E.4 Failure Modes and Footprint

Table A13 lists the controller’s behaviour by failure mode: every mode resolves to a block, a preservation, or a rollback, and none resolves to a downgrade. Table A14 gives the observed footprint at 200 services.

Table A13. Fail-closed behaviour by failure mode.

Failure mode	Detection point	Controller behaviour
Unsupported endpoint	Compatibility filter	Block assignment
Stale capability fact	Provenance validation	Block promotion
Contradictory fact	Consistency check	Block promotion
Policy conflict	Policy filter	Use the strongest valid policy, or block
Short graph outage	Reconciliation start	Keep the last committed configuration
Prolonged graph outage	Freshness window / work queue	Pin active status, block new promotions, resync from snapshot on recovery
Mesh-readiness mismatch	Health-gated rollout	Roll back or keep pending
Rapid capability churn	Hysteresis check	Keep the current profile unless the score gap exceeds δ
Stale revocation or attestation fact	Provenance validation	Block any profile requiring that evidence

Table A14. Observed controller footprint at 200 services (archived run).

Component	CPU (vCPU)	Memory (MB RSS)
Controller at 200 services	<1	<600

E.5 Positioning Against Service-Computing Research Lines

Table A15. Positioning against recent service-computing literature, by decision variable.

Research line	Main decision variable	QuantumTrustKG distinction
Kubernetes / service-level objective orchestration	Placement, scaling, traffic	Per-edge cryptographic profiles
Microservice adaptation	self- Runtime configuration	Fail-closed cryptographic admissibility
Security and authorization	Identity and access decisions	Protocol strength and post-quantum migration
Semantic/model-driven tooling	Compliance and monitoring	A focused crypto-policy ontology
Quantum-service engineering	Quantum workload deployment	Protection of classical service communication

F Expanded Threats to Validity

This section expands the threats summarized in Sec. 10.

Workloads and topology discovery. The workloads are controlled rather than production traces, and topology discovery is annotation-driven (quantumtrustkg.io/calls). A deployment that substituted telemetry-driven discovery—Envoy access logs, OpenTelemetry traces, Kiali, or mesh access logs—would exercise the controller’s reconciliation under input drift that the

current testbeds avoid. Such telemetry should be treated as provenance-qualified evidence rather than as authority, because false positives and stale traces could otherwise create edges that policy never intended to manage. The real-topology results of Sec. 8.5 address only the *structure* of a production system rather than its operational drift: the Alibaba trace supplies real caller-to-callee edges, but the cryptographic requirements over those edges are assigned by the same calibrated model as the synthetic meshes, because the trace carries no cryptographic policy. Moreover, our subgraphs are hub-centred breadth-first neighbourhoods and are therefore more concentrated than the full trace, which is why we report the measured points without extrapolating the size trend.

Cost modelling. Post-quantum setup costs are model-calibrated. The controller actions, ingestion, ranking, and mesh materialization are real. The per-handshake cost on top of mTLS establishment is measured on stock OpenSSL, but the certificate, rollout, and configuration terms that make up most of the reported 8–15% are model parameters rather than measurements. Section 8.6 gives the derivation and Table 8 marks each input, separating the measured and modelled portions. Stock Envoy/Istio output should be read as strict-mTLS materialization plus selected-profile metadata rather than as native post-quantum negotiation.

Baselines. The baselines are meaningful but neither exhaustive nor authoritative. All four competing systems are our re-creations of published decision rules (Sec. 8.4), only the QTPO-NoKG ablation is a fully controlled comparison, and it shows the smallest gap. We also do not compare against a global-optimization solver, because predictable controller latency and explainable failure are design goals rather than incidental properties. An optimizer would change what failure means rather than just how good the answer is.

Semantic validation. The semantic validation path relies on explicit Go checks over SPARQL query results rather than a fully integrated OWL/SHACL runtime. This weakens the auditability story relative to a shape-validated pipeline: the constraint that the controller enforces and the shape that the artifact ships are kept in correspondence by construction rather than by execution. Integrating SHACL is straightforward and is left to future work.

Metric construct. Because declared-label policy compliance is computed from the same declared capability and policy labels that drive the decision, it cannot distinguish a correct decision from a decision that is correct *with respect to a false declaration*. The absolute compliance figure therefore does not establish the cryptography carried on those connections. The mass-staleness sweep (Table 7) and the fault-injection suite (Table 13) both show that when labels degrade, QTPO loses availability without unsafe promotions, whereas a provenance-blind selector produces unsafe promotions in proportion to the corruption. These experiments characterize control-plane behavior under degraded inputs rather than end-to-end cryptographic behavior.

Single-cluster scope. Every measurement uses one cluster. Section 10 discusses multi-cluster behaviour from the architecture and Thm. 2 without presenting it as a result. The main missing experiment is a federated evaluation across varying cross-cluster edge fractions and controlled network partitions.